

四川大学

高等学历继续教育本科生毕业论文（设计）



学习类型 ☐ 网络教育 ☐ 成人教育 ☒ 自学考试

助学单位 四川大学

题 目 自然语言交互的企业智能商旅系统设计与实现

专 业 计算机科学与技术

学生姓名 彭军林

准考证号 010122394863

指导教师 于中华

教育培训部学历教育办公室制表

二〇二六年 四 月 一 日

自然语言交互的企业智能商旅系统设计与实现

专业：计算机科学与技术

学生：彭军林

指导教师：于中华

摘 要

传统的企业商旅管理系统存在流程割裂、合规性、成本控制难以及用户体验差等问题。本系统将微服务架构与自然语言大模型相结合，通过聊天对话方式设计并实现企业商旅智能系统。本文先通过梳理微服务架构以及自然语言交互方式在垂直领域应用的研究现状，明确了本系统的研究定位与技术路线。在此之后，详细阐述了微服务、自然语言处理、大模型智能体及商旅业务流程等相关理论与关键技术。智能商旅系统以企业商旅管理场景为应用对象，完成了系统功能性与非功能性需求分析，在此过程中设计了由展现层、网关服务层、业务微服务层、数据层构成的总体架构，并且完成了微服务拆分、数据库及向量知识库存储设计。本文重点对自然语言交互核心对话服务模块和 RAG 知识库增强检索服务模块进行详细设计，实现了基于大语言模型的自然语言理解、聊天会话和状态管理、企业差标和差旅政策的校验及跨服务调用功能。智能商旅管理系统基于 Spring Cloud 微服务生态与通义千问 API 调用的方式来完成系统开发与集成，采用 Docker 容器化部署并开展测试与评估。经过设定测试用例和系统性能验证结果表明，智能商旅系统完全可通过多轮自然语言对话完成机票、酒店、火车票等商旅订单的智能生成与合规性校验，它在任务完成率、交互效率及用户体验上较传统表单模式具有明显优势，我觉得自然语言交互为企业商旅管理的智能化、轻量化升级提供了可行方案。

关键词：微服务架构 大语言模型 自然语言交互 对话式订单 智能商旅

目 录

自然语言交互的企业智能商旅系统设计与实现.....	II
目 录.....	III
1 绪论.....	1
1.1 开发背景与意义.....	1
1.1.1 企业差旅管理现状与痛点	1
1.1.2 自然语言交互的发展趋势.....	2
1.1.3 本系统开发的价值和意义.....	2
1.2 国内外研究现状	3
1.2.1 微服务架构研究现状.....	3
1.2.2 对话系统在垂直领域的研究现状	4
1.2.3 现有研究不足与本系统的定位	5
1.3 主要研究内容与章节安排	6
1.3.1 主要研究内容.....	6
1.3.2 论文结构安排	6
2 相关技术基础	7
2.1 微服务开发技术	7
2.1.1 Spring Cloud Alibaba.....	7
2.1.2 Nacos 配置与服务注册	8
2.1.3 Spring Cloud Gateway	8
2.2 自然语言处理与 AI 技术.....	8

2.2.1 大语言模型概述	9
2.2.2 通义千问模型	9
2.2.3 提示词工程	9
2.3 搜索增强 RAG 技术.....	10
2.3.1 RAG 架构原理	10
2.3.2 向量数据库与 pgvector.....	10
3 系统需求分析	11
3.1 系统功能性需求	11
3.1.1 系统管理功能需求.....	11
3.1.2 出行管理功能需求	13
3.1.3 智能管理功能需求	15
3.1.4 数据看板功能需求	16
3.2 系统非功能性需求	17
3.2.1 高并发与可扩展性	17
3.2.2 系统高可用性	17
3.2.3 权限与数据安全.....	18
3.2.4 响应时间要求	18
3.3 本章小结	19
4 系统总体设计	19
4.1 架构设计	19
4.1.1 技术栈选型.....	19
4.1.2 逻辑架构分层	20

4.1.3	微服务拆分方案	22
4.1.4	服务治理策略	23
4.2	数据库设计	25
4.2.1	数据库选型依据	26
4.2.2	数据库结构设计说明	26
4.2.3	业务数据库 ER 图	31
4.2.4	数据库表结构设计	33
4.2.5	向量数据库扩展说明	45
4.3	核心模块划分	47
4.3.1	用户服务模块	47
4.3.2	订单服务模块	48
4.3.3	流程审批模块	48
4.3.4	渠道服务模块	49
4.3.5	对话服务模块	50
4.3.6	RAG 服务模块	50
4.4	本章小结	50
5	系统详细设计与实现	51
5.1	对话服务核心实现	51
5.1.1	通义千问 API 接入与封装	51
5.1.2	自然语言理解实现	53
5.1.3	多轮对话管理实现	55
5.1.4	服务调用实现	56

5.1.5 对话回复生成实现	57
5.1.6 RAG 知识增强实现	58
5.2 业务微服务实现要点.....	59
5.2.1 用户服务	59
5.2.2 订单服务	59
5.2.3 审批服务	59
5.2.4 渠道服务	59
5.3 系统界面展示	60
5.3.1 对话下单界面	60
5.3.2 订单管理界面	60
5.3.3 审批流程界面	61
5.5 本章小结	62
6 系统测试	62
6.1 部署架构与测试环境.....	62
6.1.1 部署架构.....	62
6.1.2 测试环境.....	63
6.2 功能测试	64
6.2.1 对话下单功能测试	64
6.2.2 订单管理功能测试.....	65
6.2.3 审批流程功能测试.....	66
6.2.4 测试结果分析	67
6.3 性能测试	67

6.3.1 接口压力测试	67
6.3.2 系统资源占用监控.....	68
6.3.3 测试结果分析	68
6.4 对话效果专项评估	69
6.4.1 测试数据集构建	69
6.4.2 评估指标	69
6.4.3 效果对比分析	70
6.4.4 评估结论	70
6.5 本章小结	71
7 总结与展望.....	71
7.1 开发工作总结	71
7.1.1 完成的主要工作.....	71
7.1.2 系统达到的目标	72
7.2 系统特色与不足.....	73
7.2.1 系统特色	73
7.2.2 存在的不足.....	74
7.3 后续改进方向.....	75
7.3.1 引入更精细的领域微调模型	75
7.3.2 支持多模态交互.....	75
7.3.3 增强复杂对话场景的处理能力.....	75
7.4 结语	75
参考文献.....	76

致 谢.....	81
----------	----

1 绪论

1.1 开发背景与意义

1.1.1 企业差旅管理现状与痛点

全球商务活动越来越频繁，企业数字化转型也在加速，企业的差旅管理正处于机遇与挑战并存的关键时期。在中国，差旅支出已成为企业除人力成本之外的第二大可控成本，这些成本甚至占企业非薪资支出的 10%-20%。企业设定不同岗位级别的差旅标准，说白了就是想平衡员工体验和成本控制。说到底，企业降本增效的要求日益变得迫切。

现在企业差旅的管理方式还是不太一样的：大多数中小企业多采用员工自行预订、事后报销的传统方式；一些大中型企业则逐步转向系统化管理，通过自建或采购差旅管理系统实现流程线上化；极少部分的企业委托专业差旅管理公司（TMC）进行集中采购。技术手段上，企业的差旅管理方式已从 Excel 表格发展到对接 OA 审批系统、上专业的商旅平台，现在连核心服务搬到手机上了。然而，传统系统开发周期长、个性化需求依赖二次开发，导致成本居高不下。

目前企业的差旅管理系统仍存在很多深层次痛点，主要体现在以下五个方面：

（1）流程割裂导致效率低下：员工进行差旅活动可能需在OA、预订平台、报销系统等多个平台间切换，反复且重复地录入差旅信息，基本上员工一次差旅活动平均耗时可能在45-60分钟，每年浪费约15-20小时（近3个工作日）[2]。

（2）政策执行难成本就失控：企业虽制定了详细的差旅标准，但在执行中难以实时管控，员工易无意或故意超标，大多数企业的审批多依赖事后审核。值得注意的是，缺乏有效管控的企业差旅成本比规范管理的企业高出20%-30%[3]。

（3）系统操作复杂影响用户体验：传统系统以管理为中心，功能繁杂、菜单层级深，移动端体验差，通常异常处理依赖人工，报销的周期也会拉长，员工满意度普遍低于40%[4]。

（4）数据孤岛决策没法优化：每家企业使用的系统可能不止一个，各系统数据分散、设计实现标准也不一样，难以形成完整规范的数据视图，在实际的应用中无法实时分析趋势和优化采购策略，超过60%的管理者表示难以准确掌握差旅支出构成[5]。

（5）合规风险突出：令企业头疼的是员工提供虚假发票、存在重复报销、拆单规避审批等现象，甚至有不合规支出占比约5%-10%，造成直接经济损失不说，审计风险也来了[6]。

上述痛点表明，企业在提升差旅管理的自动化、智能化和精细化水平上还有待提高，我们可以考虑积极引入AI技术以实现降本增效。

1.1.2 自然语言交互的发展趋势

自然语言交互不再只是一个辅助功能，它已经慢慢变成企业应用的核心。变化主要在这三个层面：

(1) 技术层面：以前是一问一答，现在能记住上下文了。大模型发展起来后，对话窗口能装下几万token，复杂业务的多轮交互也就成了现实。有些场景还在尝试语音、图像辅助，聊起来更自然。[4]

(2) 应用层面：不光是能问答了，还能跑业务流程、管知识库、辅助决策。用户说句话就能完成预订、审批、查询——这就是“对话即操作”。RAG和工具调用现在基本是标配，知识更新慢和模型幻觉的问题也缓解了不少。[5]

(3) 架构层面：以前靠规则和意图识别，现在是大模型驱动的智能体。多智能体协同、长期记忆、个性化学习这些能力越来越成熟，对话系统从“助手”变成了“同事”，能理解你想干啥，然后调对工具来处理。[6]

估计未来3-5年，“AI助手+传统界面”的混合模式会普及，自然语言交互会成为企业系统的标配。顺着这个思路，我们做了这个智能商旅系统，想把上面的趋势落地成实际的东西。

1.1.3 本系统开发的价值和意义

传统流程下，员工订一次差旅，光预订就得 30 分钟，审批又要 15 分钟，加起来平均 45 分钟。

换成 AI 对话下单后，系统会引导用户一步步填信息，大概 1 分钟；然后 AI 自动推荐方案，用户确认一下，约 2 分钟。如果对初选方案不满意，换个方案再加 2 分钟。审批环节，AI 自动做合规校验，符合政策的订单秒过，审批时间基本可以忽略不计。

算下来，正常情况下一趟差旅约 3 分钟，就算要改选也就 5 分钟，比传统方式快了 8-9 倍。拿一个 1000 人的公司来说，假设每人每年出差 3 次，一年就是 3000 次差旅。按 30%的订单需要改选来加权平均，每年能省下约 2070 小时。按一年 2000 个工时算，差不多相当于省出 1 个人力。

针对差旅管理中效率低、成本失控、体验差这几个老问题，我们做了这个智能商旅系统。它的价值主要有四点：

(1) 效率上，重构了预订流程

把原来“申请-预订-审批-报销”这套多系统流程，变成了一次对话就搞定。前面算了，单次差旅从 45 分钟压到 3-5 分钟，一年能省 2070 小时，相当于多出一个人干活。

(2) 成本上，把合规校验提到了事前

以前是事后审核，现在对话过程中实时校验差旅标准，违规操作当场拦截。有研究说，管得松的企业差旅成本比管得严的高出 20%-30%[3]。我们这个系统通过事前合规校验，预计能降 15%-25%的成本，财务审核的工作量也能减不少。

(3) 技术上，验证了大模型在企业落地

我们用 RAG 架构把企业私域政策和通用 AI 能力结合起来，既保障了数据安全，又提升问答效果。微服务设计让 AI 对话和业务系统解耦，给传统企业接入 AI 能力打了个样。

(4) 行业价值上，推动了人机协作的转变

这个系统让企业软件从“人适应系统”变成“系统适应人”，财务、HR 等其他领域要搞 AI 改造也能参考。只有员工能用最自然的方式跟系统对话，企业数字化转型才算真正做到了以人为本。

总的来说，这个智能商旅系统给差旅管理的老大难问题提供了一个新解法，在实现自动化、智能化、精细化管理的同时，也跟上了 AI 技术变革的步伐。

1.2 国内外研究现状

1.2.1 微服务架构研究现状

微服务架构现在已经是企业应用开发的主流方式了，很多实际系统都在用。说白了，就是把一个大单体应用拆成一堆小服务，每个服务只管一件事，互相之间松耦合，能单独跑、单独部署。这样开发起来快，想改哪块就改哪块，测试也方便。不少学者也从架构演进、服务治理、系统迁移这些角度做了不少研究[7][49]。

一、微服务架构怎么演进

微服务转型这事挺复杂的，一口气全改不现实。Lewis 和 Fowler 等学者总结了按步走的方法，一般分四步走[7]：

- 第一步，先把单体优化好。代码模块化搞干净，数据库也解耦。
- 第二步，试着做微服务化。开始拆独立服务，但先不拆太狠，保留一些共享库。
- 第三步，进入真正的微服务阶段。每个服务有自己的数据库，治理能力也

跟上。

- 第四步，上云原生。用容器部署、Service Mesh、服务编排这些。这个渐进方式国内外不少企业都验证过，确实能降低转型风险。

二、主流微服务方案有哪些

现在主流的微服务实现方案，大概有这么几类：

- Spring Cloud 生态：基于 Spring Boot，服务治理能力很全。国内大中型企业用得最多。优势是生态成熟、上手不难，适合 Java 技术栈的公司。
- Dubbo 高性能 RPC：阿里开源的 RPC 框架，主打高性能通信。适合高并发、内部调用特别频繁的互联网场景。
- Service Mesh 云原生方案：Istio、Linkerd 这些为代表。把服务治理能力下沉到基础设施层，跟业务代码解耦。适合多语言混合的系统，或者跑在 K8s 上的环境[13]。
- Kubernetes 原生方案：直接用 K8s 自带的能力(Service、ConfigMap、Ingress)来做微服务治理。技术栈简单，适合已经全面容器化的企业。

三、微服务最近在研究什么

最近微服务这块的研究，有几个方向比较热：一是跟云原生越绑越深，容器化部署和 K8s 编排基本成了标配。二是服务治理能力一直在加强，可观测性、安全性、韧性这些是重点。三是 Serverless 慢慢起来了，运维复杂度降了不少。

这些研究给我们智能商旅系统提供了不少理论基础。

1.2.2 对话系统在垂直领域的研究现状

对话系统的核心目标，就是让计算机能跟人用自然语言聊天。深度学习和大语言模型发展起来后，垂直领域的相关应用研究也越来越深入，成了学术界和工业界都关注的热点。

(1) 对话系统是如何一步步发展过来的

对话系统的研究可以追溯到 20 世纪 60 年代的 ELIZA 系统。从那以后，先后经历了基于规则的符号系统、基于统计学习的对话管理，以及基于深度学习的端到端模型这几个阶段[14]。按研究侧重点来分，主要有任务导向型和非任务导向型两大类。其中任务导向型系统能完成预订、查询这类具体任务，所以成了垂直领域的主流研究范式。

最近几年，BERT、GPT 这些预训练语言模型的突破，让对话系统发生了很大的变化[38]。研究范式从“预训练+微调”转向了“提示学习”，对标注数据的依赖明显降低了[15]。提示词工程的研究也表明，结构化的提示模板能有效提升特定领域的任务完成率[16]。这些进展为对话系统在垂直领域的落地打下了不错的底

子。

(2) 垂直领域对话系统的研究现状

国内外的学者在垂直领域做了不少工作，下面按领域梳理一下：

- **医疗健康领域：**这是比较热的一个方向。王等人用分层强化学习搭了个三层架构的医疗问诊系统，能主动引导对话[17]；刘等人把医学知识图谱融进去，诊断准确率有所提升[18]；赵等人还尝试用多模态情感识别来做心理疏导[19]。
- **金融领域：**主要聚焦智能推荐和文档处理。陈等人搞了个三级推荐架构，能比较好地解析模糊需求[20]；周等人做的金融文档结构化抽取，准确率能达到 99%以上[21]。
- **政务领域：**孙等人基于 RAG 搭建了政务问答系统，解决了知识更新慢的问题[23]；吴等人用对话状态跟踪技术，提升了复杂政策的解读效率[24]。
- **智能客服领域：**黄等人系统梳理了意图识别、槽位填充这些核心模块的演进[25]；杨等人提出的知识图谱双塔架构，显著提高了问题解决率，转人工率也降下来了[26]；钱等人基于元学习的方法，降低了跨领域迁移的成本[27]。

1.2.3 现有研究不足与本系统的定位

现在市面上大部分企业商旅服务商面临 AI 集成难、服务同质化、流程繁琐这几个问题都还没解决好。很多方案还是“传统平台+多端服务”的老方法，主要靠人来补位。就算有些企业用了 AI，也基本停留在写代码时帮忙补全的层面，没真正想清楚 AI 怎么给产品赋能。

自然语言交互是 AI 垂直应用的一个重要方向，优势很明显。用户说句话就能完成需求描述和订单生成，从交互到交易一步到位。不过挑战也不少：意图理解要准、数据安全要保、真实场景要适配、提示词要动态调、个性化能力要跟上。这些都还得花功夫去解决。

针对上面说的这些问题，我们做了这个智能商旅系统——基于微服务架构和大语言模型，用自然语言对话来交互。就是想探索一下，AI 到底怎么跟商旅管理深度结合。

1.3 主要研究内容与章节安排

1.3.1 主要研究内容

这篇论文主要做的是基于自然语言对话交互的智能商旅系统，具体工作分四个方面：

(1) 微服务架构搭建：用Spring Cloud Alibaba这套技术栈，把服务注册发现、API网关、服务间通信、熔断限流这些治理机制都搭起来。然后按业务拆出了用户服务、订单服务、对话服务等几个微服务，让它们能各司其职、高效配合。

(2) 对话式订单生成：设计了一套基于提示词的方案来做意图识别和槽位提取，把用户说的自然语言转成结构化的订单信息。同时用Redis存会话上下文，实现多轮对话的状态管理，保证对话不中断、不乱套。

(3) RAG知识增强：建了一个企业差旅政策的向量库，用RAG技术把私域知识“喂”给大模型，用来做政策合规校验和智能问答。这样能在很大程度上缓解模型胡编乱造的问题。

(4) 系统集成与效果评估：把对话服务和业务微服务接在一起，然后做了功能测试、性能测试和对话效果专项评估，验证这套系统到底能不能跑通、好不好用。

1.3.2 论文结构安排

全文共分七个章节，安排如下：

第1章 绪论：先讲研究背景和意义，分析企业差旅管理现在有什么问题、痛点在哪，再梳理一下自然语言交互的发展趋势，最后说明研究内容和论文结构。

第2章 相关技术基础：介绍系统开发用到的几项关键技术——Spring Cloud Alibaba微服务框架、通义千问大模型、提示词工程，还有RAG检索增强生成。

第3章 系统需求分析：从功能需求、非功能需求、自然语言交互需求三个角度来分析，把系统边界和性能指标定清楚。

第4章 系统总体设计：说清楚系统架构怎么设计、数据库怎么建、模块怎么划分，再把技术栈选型和微服务拆分方案确定下来。

第5章 系统详细设计与实现：这部分写得比较细，包括微服务基础环境怎么搭、业务微服务怎么实现，还有对话服务核心模块的具体实现——API接入、自然语言理解、多轮对话管理、服务调用、回复生成、RAG知识增强等。

第6章 系统测试：设计功能测试、性能测试和对话效果专项评估方案，用测试数据来验证系统功能全不全、性能达不达标。

第7章 总结与展望：总结全文工作，分析系统的特色和不足，再说说后续可以从哪些方向继续改进。

2 相关技术基础

本章对系统开发所涉及的关键技术进行介绍，包括微服务开发技术、自然语言处理与AI技术以及检索增强生成技术。这些技术共同构成了系统的技术底座，为后续需求分析、系统设计和实现提供理论支撑[37]。

2.1 微服务开发技术

前面1.1.1节提到，传统差旅管理系统流程割裂、操作复杂，本质上是因为单体架构把各功能模块绑得太紧，想改一块就得动全身。微服务架构正好能解决这个问题——把一个大应用拆成多个小服务，每个服务只管自己的业务，独立开发、独立部署、独立扩展[41][49]。

我们的系统用Spring Cloud Alibaba这套技术栈来搭微服务。下面挨个介绍下用到的核心组件，以及它们在本系统里具体怎么用的。

2.1.1 Spring Cloud Alibaba

Spring Cloud Alibaba 是阿里开源的一个微服务解决方案[37]，用起来比较省心，基本上一站式配齐。它是 Spring Cloud 生态的一部分，把阿里多年搞微服务的经验打包好了，直接提供服务注册发现、配置管理、限流降级、分布式事务这些核心能力。

对比一下 Spring Cloud Netflix（现在已经进入维护模式了），Spring Cloud Alibaba 有几个明显的优势：一是组件一直在更新，跟 Spring Boot 最新版能保持同步；二是功能更全，Nacos、Sentinel 这些成熟组件都内置好了；三是中文文档很全，国内企业用得更多。

我们选 Spring Cloud Alibaba 来搭微服务，主要是看中它生态比较完整、社区活跃，而且跟我们现有的技术栈能很好地配合。

2.1.2 Nacos 配置与服务注册

Nacos 是阿里开源的服务注册和配置中心，名字拆开看就是“Naming”加“Configuration”。

（1）服务注册与发现

微服务架构下，服务实例的地址是动态变的，随时可能换地方。Nacos 解决的就是“服务在哪儿”这个问题。服务启动时把自己（IP、端口、健康状态等）报到 Nacos，消费者需要调谁就去 Nacos 查，拿到实例列表再走负载均衡[37][41]。我们系统里所有微服务都注册到 Nacos，互相之间动态发现、动态调用。

（2）配置管理

Nacos 可以把配置集中存起来，支持版本管理和动态刷新。改了配置不用重启应用，直接生效。我们用 Nacos 来管数据库连接参数、通义千问的 API 密钥、差标规则阈值这些东西。不同环境（开发、测试、生产）的配置隔离开，改完动态刷新，很方便。

2.1.3 Spring Cloud Gateway

Spring Cloud Gateway是Spring官方基于WebFlux做的API网关，负责请求路由、认证鉴权、限流熔断这些事。它的核心就三个东西：路由、断言、过滤器。跟老一代的Zuul网关比，Gateway底层用的是Netty的异步非阻塞模型，吞吐量更高。

我们系统用Gateway主要干三件事：

- **统一入口：**所有客户端请求都先打到网关，再由网关转发到对应的微服务。
- **JWT认证：**在网关层解析和校验Token，然后把用户信息（userId、角色）塞到请求头里，透传给下游服务。这样各个微服务就不用重复写认证逻辑了。
- **路由配置：**根据请求路径，动态路由到用户服务、订单服务或对话服务。

2.2 自然语言处理与 AI 技术

前面1.1.1节提到，传统系统效率低、体验差，本质上是“人适应系统”的交互模式。现在大语言模型发展起来了，让“系统适应人”变成了可能。所以我们接了通义千问大模型，用提示词工程来做商旅场景的意图识别和订单生成。

2.2.1 大语言模型概述

大语言模型（LLM）简单说就是参数规模很大的神经网络语言模型，几十亿到几千亿参数不等。2018年BERT出来后，LLM从“预训练+微调”慢慢转向了“提示学习”。2022年ChatGPT出来，对话这块算是真正能用了。

跟传统的NLP方法比，大模型的优势很明显：不用大量标数据，写写提示词就能干活；而且能记住上下文，多轮对话不跑偏。现在主流的商业大模型有GPT系列、Gemini、Claude、通义千问这些。我们系统用的是通义千问。

2.2.2 通义千问模型

通义千问是阿里做的大语言模型，中文理解、代码生成这些方面表现不错。它有好几个版本：Qwen-Turbo、Qwen-Plus、Qwen-Max。我们选了Qwen-Plus，主要是这么考虑的：

- 商旅场景对响应速度有要求，Qwen-Plus的延迟能接受；
- 订单生成、信息提取这些任务不算太复杂，没必要上最强版本；
- API定价合理，适合开发和部署。

通义千问支持文本生成、多轮对话上下文维护、工具调用（Function Calling）、结构化输出（JSON）这些能力。我们主要用了它的多轮对话和工具调用，来做智能商旅助手的核心功能。

2.2.3 提示词工程

提示词工程就是通过设计合适的输入提示词，让大模型按我们想要的方式输出[46][50][52]。模型微调成本高的时候，提示词工程就成了LLM快速落地的好办法。

做好提示词，一般要注意这几点：给模型设定一个角色（比如“你是个专业的商旅助手”）；把任务说清楚；规定输出格式（比如JSON）；给几个例子；划清边界限制。

按使用场景分，有零样本提示、少样本提示、思维链提示这些类型。

我们系统在订单生成任务里，用了固定的系统提示词模板，把商旅助手的角色、企业差旅标准、JSON输出格式这些都打包进去。这样模型输出的稳定性和合规性都提高了不少。具体的提示词模板在第5章5.3.2节会详细说。

2.3 搜索增强 RAG 技术

前面1.1.1节提到，差旅政策执行难是企业的一大痛点。传统做法是把政策规则硬编码到系统里，政策一改就得动代码、重新部署，反应很慢。大语言模型虽然能理解自然语言，但它训练时没看过企业内部的私有政策，直接问的话要么瞎编，要么答错。

RAG技术通过接入外部知识库，让模型能根据企业实际的政策文档来回答，正好解决了这个问题。我们系统就用RAG搭了企业差旅政策知识库，用来做政策合规校验和智能问答。

2.3.1 RAG 架构原理

RAG架构分两块：检索器和生成器。检索器负责从知识库找跟用户问题相关的文档片段，生成器把找到的内容和原问题拼一起，再扔给大模型生成答案[52]。

RAG的好处主要有三点：

- **知识更新快：**知识库跟模型是分开的，不用重新训练模型；
- **来源可查：**回答能对应到具体文档；
- **成本低：**花小钱就能实现知识增强。

我们系统把企业差旅政策文档向量化后存进知识库，用户提问时先检索相关政策片段，再注入大模型生成回答，用来做政策合规校验。

2.3.2 向量数据库与 pgvector

向量数据库说白了就是用来存和查高维向量的[52]。pgvector是PostgreSQL上的一个开源向量检索扩展，支持存浮点数向量，能做欧氏距离、余弦距离这些相似度计算，还提供了IVFFlat和HNSW两种索引来加速检索。

我们选pgvector做向量数据库，主要这么考虑的：

- 不用额外装别的，直接用现有的PostgreSQL就行；
- HNSW索引检索快，能满足实时问答的需求；
- 跟Spring Data JPA这些技术栈配合得好，开发成本低。

pgvector有IVFFlat和HNSW两种索引可选。我们选了HNSW，因为它查询精度更高、检索更快，更适合实时问答场景。

我们把企业差旅政策文档分块向量化后存进pgvector。用户提问时，用余弦

相似度去检索匹配的政策片段，默认返回相似度最高的3-5条。这个数量是根据经验调的：太少了（1-2条）可能漏掉关键政策，太多了（6条以上）会引入无关信息，还费token。检索结果作为上下文喂给大模型，实现知识增强的智能问答。

3 系统需求分析

本章从功能性需求开始系统性设计实际问题，然后通过非功能性需求对智能商旅系统将要达成的功能性能等指标进行阐述，为后续智能商旅系统设计提供参考依据。

3.1 系统功能性需求

前面说的几个痛点，对应到系统上要解决三块：组织架构管理、出行流程管控、智能化能力支撑。功能性需求分了四个模块：系统管理（企业、部门、员工、用户）、出行管理（订单、差标、审批）、智能管理（RAG知识库、会话）、数据看板（统计、监控），各模块功能具体如下。

3.1.1 系统管理功能需求

用户出差时大多没有固定工位，普通功能放移动端。管理者则用后台系统。

3.1.1.1 企业管理

这个模块主要解决前面1.1.1节说的“数据孤岛”问题，通过企业ID把不同租户的数据隔离开。

企业管理这块，需要支持企业基本信息维护，主要包括：企业名称、管理账号、密码、是否开启自动审批、金额超过多少必须走审批等配置项。

管理账号和密码由系统初始化生成，企业管理员直接使用，省去了自己设定的麻烦。企业订单流程是否需要审批，也从企业配置里读取，方便统一控制。

企业ID会作为全局数据隔离的依据，整个系统的权限和数据过滤都依赖它。

3.1.1.2 部门管理

这个模块主要解决前面1.1.1节说的“数据孤岛”问题，通过部门树形结构实现分部门的数据隔离和权限控制。

部门是企业组织架构的基本单元，上下级关系通过父部门ID来关联。部门管理不用搞太复杂，不需要维护复杂的节点信息，只需设定部门名称和备注就行。实际展示时，用树形结构来呈现部门层级，这是比较通用的做法。

设计成这样，主要是为了适应企业自身的组织管理需求，方便界定不同部门的角色分工。

3.1.1.3 员工管理

这个模块主要把用户账户和组织架构关联起来，还要支持同一用户在多个企业任职的情况。

员工是企业组织架构中的具体人员，需要记录一些基本信息来标识身份，比如：姓名、职位、工号、所属企业、所属部门、用户ID等。

考虑到同一个人可能在多个企业上班，系统允许同一个用户ID关联多个企业的员工信息。用户登录时，系统根据用户ID就能判断他当前在哪个企业、哪个部门。

3.1.1.4 用户管理

这个模块主要负责登录账户和权限控制，保障账户安全，解决前面1.1.1节说的“合规风险”问题。

（1）用户管理

用户是系统的基础，所有数据都通过用户关联起来，可以理解为“人”在系统中的映射。用户信息包括用户名和加密后的密码，保证账号安全。

用户管理提供基本的账户维护功能，还包括验证码（防止暴力破解和恶意登录）、注册、登录、退出等。用户登录后，系统会从用户信息里拿到对应的员工信息，从而识别出他所在的企业和部门。登录状态会维持一段时间，会话超时后需要重新认证。

（2）注册方式

本系统不开放公开注册，采用企业管理员统一开通账户的模式。管理员在后台创建员工账户后，员工通过手机号或邮箱激活。演示环境下，系统提供一个默

认企业账户供体验使用。

(3) 角色权限

角色分四类：超级管理员、管理员、审核员、普通用户。角色直接赋给用户，不用搞复杂的关联关系。

- **超级管理员**：管所有企业和整个系统，能操作所有管理功能。
- **管理员**：企业级的最高权限管理者。
- **审核员**：仅次于管理员，有订单审批权限。
- **普通用户**：只能使用业务功能。

使用端上，超级管理员、管理员、审核员同时有后台管理系统和移动端的权限；普通用户只有移动端权限。

3.1.1.5 权限控制

为了解决前面1.1.1节说的“合规风险”问题，系统做了两方面的权限控制：

(1) 数据隔离

- **普通员工**：只能看和操作自己的订单
- **审批人**：只能看本部门及下属部门的待审批申请
- **管理员**：拥有所属企业的全部权限
- **超级管理员**：拥有全系统权限

(2) 操作审计

- **敏感操作**（比如改政策配置、改用户权限）会记日志，包括谁干的、什么时候干的、干了什么，方便以后查。

3.1.2 出行管理功能需求

出行管理是系统的业务核心，主要做三件事：

- **订单管理**：把机票、酒店、火车、用车这四类出行数据统一归拢
- **差标管理**：对各出行场景设定金额限制，用于成本控制
- **审批管理**：处理需要审批的场景以及超标的订单

3.1.2.1 订单管理

这个模块主要解决前面1.1.1节说的“流程割裂”问题，把原来分散的申请、预订、审批串成一条统一的流程。

(1) 订单数据来源

订单数据从移动端产生，最终归到订单管理模块里，移动端也能查看。管理端没有新增订单的功能，也不能修改订单。

(2) 订单信息结构

订单包含以下信息：订单号、类型（机票、酒店、火车、用车）、金额、订单状态、审核状态、行程信息等。每个订单关联一段行程，行程包括出发地、目的地、出发时间、到达时间。

最终的出票走第三方渠道，票务信息和具体行程乘客关联。

(3) 订单状态流转

用户提交订单后，系统会检查企业配置的差标和审批规则：

- 符合差标且企业开了自动审批 → 订单自动通过，进入渠道预订
- 需要人工审批 → 生成审批记录，订单状态变为“待审批”
- 违反差标 → 系统提示违规原因，阻止提交

审批通过后，订单进入渠道预订；第三方渠道预订成功后，状态变为“待支付”；审批驳回后，订单标记为“已取消”。订单状态只能往前走，不能回退（比如已审批的不能退回草稿），这样流程才够严肃、可追溯。

订单支付和退款功能属于商业扩展模块，不在本系统的研究范围内。订单状态可以扩展出已支付、已完成、已取消、退款中、已退款等。

3.1.2.2 差标管理

这个模块把企业差旅政策转成程序能校验的规则，解决前面1.1.1节说的“政策执行难”问题。

差标管理按企业、部门等维度来设置。每个差标有个名称，方便理解它是干什么用的。差标的具体限制按业务场景来定，比如：

- **使用场景：**机票上限、酒店上限、火车上限、用车上限
- **时间跨度：**月度上限、年度上限

校验规则

用户提交订单时，系统根据他所属的企业和部门，匹配对应的差标来做校验。如果订单金额超过了对应维度的上限，系统就阻止提交，并提示超标原因。

3.1.2.3 审批管理

这个模块负责订单审批流程，让合规订单快速过，违规订单走人工处理。

(1) 审批入口

审批操作可以在后台管理系统和移动端进行。企业管理员和审批员可以点“通过”或“拒绝”。

(2) 什么情况走审批

- 企业配置了审批流程（不是自动审批模式）
- 订单金额超过了企业设定的审批阈值
- 订单违反差标，需要特批

(3) 审批规则

- 审批人只能批本部门及下属部门的订单
- 审批通过后，订单进入渠道预订环节
- 审批未通过，订单标记为已取消
- 每次审批要记下操作人、操作时间、审批意见，方便追溯

3.1.3 智能管理功能需求

我们用RAG技术让通义千问大模型能理解企业的差旅政策。智能管理模块主要支撑这么一套流程：

(1) 文档上传与解析

支持Pdf、Word、TXT格式的政策文档上传。系统自动解析内容，按每块约500字符切分。

(2) 向量化存储

调用通义千问的Embedding接口，把文本块转成1536维向量，存到pgvector向量库里。

(3) 相似度检索

用户提问时，系统把问题也转成向量，去库里找最相关的3-5个政策片段。

(4) 知识增强问答

把找到的政策片段作为背景信息喂给大模型，让它按企业政策来回答。

智能管理模块提供三个子功能：文件记录、差旅政策管理、会话管理。下面挨个说明。

3.1.3.1 文件记录

这个模块负责记录企业上传的政策文件，方便追溯知识库的变更历史。

企业可以上传多种格式的知识库文件。每次上传都会记录下来（包括原始文件名），不允许删除或修改，防止知识库内容被恶意篡改或误删。每个上传的文件都会作为向量化知识库的数据来源。

3.1.3.2 差旅政策

这个模块主要做两件事：

- **向量化存储：**把企业上传的差旅政策文档转成向量存起来
- **政策检索：**支持按需检索政策内容

企业从后台管理系统上传知识库文件后，系统会保存为向量化记录，并提供查询和详情查看功能。

3.1.3.3 会话管理

这个模块负责记录用户和 AI 助手的完整聊天记录，方便排查问题和分析对话质量。会话管理的具体功能如下：

- **会话创建：**用户第一次进聊天窗口时，系统自动建一个会话，并生成标题
- **意图识别：**对话过程中识别用户想干什么（下单、咨询、闲聊），决定后面怎么处理
- **问题澄清：**AI 看不懂用户意图或信息不够时，主动反问用户
- **消息记录：**每条对话（用户说的和 AI 回的）都关联到对应的会话
- **方案推荐记录：**系统推荐的出行方案要跟对话消息关联保存
- **用户选择记录：**用户从推荐方案里选了哪个，也要记下来，方便以后查
- **管理端查询：**管理员可以按企业、用户、时间范围查会话历史

3.1.4 数据看板功能需求

解决1.1.1节的“数据孤岛”问题。面向管理员和审批人，用图表展示差旅运营数据，方便他们实时掌握支出趋势和审批效率。

3.1.4.1 订单统计

后端数据看板需要按企业维度统计订单，包括：订单总数、待审订单数、本月订单数量。单独做一个订单统计接口就行，查询时用缓存来提升效率。

3.1.4.2 功能地图

后端需要加一个快速跳转功能，可以跳转到：企业管理、部门管理、员工管

理、用户管理、订单管理、流程审批。

3.1.4.3 核心数据展示

数据看板核心区域需要展示两块内容：

- 最新订单
- 待审批订单

这样管理人员能及时了解订单流转情况和待审批数量。点击可跳转到对应的管理页面进行具体操作。

3.2 系统非功能性需求

非功能性需求方面，系统质量指标主要从以下几个维度来定：

- 高并发与可扩展性
- 系统高可用性
- 数据安全与权限控制
- 响应时间要求

3.2.1 高并发与可扩展性

（1）需求背景

差旅预订有明显的波峰波谷：节假日前3天、周一上午9-11点，访问量特别大。系统要是扛不住，请求就会超时或者直接崩掉，员工就订不了票。

（2）指标要求

根据某中型企业（约2000人）的数据，高峰时QPS大概在200-300。考虑到系统是多租户的，还得留点业务增长的空间，我们按3-5倍的冗余来算，要求系统能支持1000QPS的并发。

（3）可扩展性要求

微服务架构下的各个组件（用户服务、订单服务、对话服务）要能独立水平扩展，加机器就能线性提升处理能力。

3.2.2 系统高可用性

（1）需求背景

差旅系统对可用性要求高，系统一挂员工就出不了差，业务损失也不小。

（2）指标要求

核心服务（对话服务、订单服务）的可用性要99.5%以上，算下来一个月不可用时间不能超过3.6小时。

关键组件得用集群部署，避免单点故障。服务注册中心、数据库这些基础设施要能自动恢复。系统还要配健康检查和自动重启，服务挂了能快速拉起来。

3.2.3 权限与数据安全

需求背景：差旅系统里有员工个人信息、财务数据、企业政策这些敏感数据，数据安全马虎不得。相关处理如下表3-1所示。

表3-1 权限与数据安全指标

安全维度	核心措施	关键说明
身份认证与授权	JWT认证、角色权限隔离、操作日志审计	Token设置有效期；普通员工仅本人订单，审批人仅本部门申请，管理员全权限；敏感操作记录日志
数据加密与传输	BCrypt密码加密、HTTPS传输、API密钥认证	密码加密存储；服务间通信采用HTTPS；API密钥禁止硬编码
数据隐私保护	信息脱敏、对话记录定期清理	身份证号、手机号等展示脱敏（如138****1234）；对话记录保留90天后自动清理

3.2.4 响应时间要求

（1）需求背景

对话快不快，直接影响用户体感。人机交互这块有研究说，用户能等的极限大概是3秒：

- 1秒以内 → 感觉“秒回”
- 1到3秒 → “稍微等了一下，还行”
- 超过3秒 → 开始烦躁

（2）指标要求

对话接口的响应时间（用户说话到系统回复）要控制在3秒以内，95%的请求不能超过3秒。订单查询、政策配置这些非对话接口，响应时间控制在1秒以内。

3.3 本章小结

这一章主要分析系统需要做什么。

(1) 功能性需求分了四个模块

- **系统管理**：通过企业、部门、员工、用户四层结构，实现数据隔离和权限控制
- **出行管理**：包括订单、差标、审批三块。订单状态按“草稿→待审批→已审批→已预订→待支付”流转；差标可按企业、部门等维度配置；审批支持后台和移动端双渠道
- **智能管理**：基于RAG技术，支撑文档上传解析、向量化存储、相似度检索、知识增强问答这一整套流程，配套文件记录、差旅政策、会话管理等功能
- **数据看板**：提供订单统计、功能地图、核心数据展示，方便管理者实时了解差旅运营情况

(2) 非功能性需求从四个维度来定

- **高并发与可扩展性**：根据差旅业务高峰特征，要求能支撑1000 QPS
- **高可用性**：核心服务可用性99.5%以上
- **权限与数据安全**：用JWT认证、BCrypt加密、数据脱敏等措施
- **响应时间**：对话接口响应控制在3秒以内

这些需求分析给第4章的系统总体设计打了个底。

4 系统总体设计

这一章主要说系统的总体设计，包括架构怎么搭、数据库怎么建、模块怎么划分[54]。

4.1 架构设计

4.1.1 技术栈选型

系统架构上做了前后端分离：

- 后端：Spring Cloud Alibaba微服务技术栈
- 前端：PC端管理后台 + 移动端H5界面（Vue[42]）

核心技术选型的依据，见表4-1。

表4-1 技术栈选型与选型依据

技术分层	技术选型	选型依据
前端框架	Vue 3	轻量级、组件化、生态完善，适合快速构建管理后台
移动端	H5 + Vant	移动端适配良好，组件库丰富，跨平台兼容
后端框架	Spring Boot 3.3.5	生态成熟，与Spring Cloud无缝集成
微服务框架	Spring Cloud Alibaba 2023.0.1.0	组件完整，社区活跃，国内企业实践丰富
服务注册与配置	Nacos	同时提供服务注册与配置管理，部署简单
API网关	Spring Cloud Gateway	响应式编程模型，性能优于Zuul
服务调用	OpenFeign	声明式HTTP客户端，与Spring生态集成良好
熔断限流	Sentinel	轻量级，控制台功能完善，支持动态规则配置
数据库	PostgreSQL	功能强大，支持pgvector向量扩展，开源免费
向量数据库	pgvector	PostgreSQL原生扩展，无需引入额外组件
缓存	Redis	高性能内存数据库，支持会话存储和分布式锁
AI模型	通义千问 Qwen-Plus	中文理解能力强，支持函数调用，API稳定
容器化	Docker	环境一致性，便于部署和运维

4.1.2 逻辑架构分层

系统分了四层：前端展示层、网关层、业务服务层、数据层。层与层之间走标准化接口通信。

（1）前端展示层

按用户角色分两块：

- **管理后台**：用Vue3 + Element Plus搭的，给管理员用，管政策配置、看数据报表、做审批等
- **移动端H5**：用Vue3 + Vant搭的，给员工用，主要就是对话式预订界面，可以嵌到企业微信、钉钉里

（2）网关层

基于Spring Cloud Gateway做的，作为系统的统一入口，干这几件事：请求路由、JWT认证鉴权、用户信息透传、限流熔断（用Sentinel）、记日志。

(3) 业务服务层

按业务边界拆成多个独立微服务，每个服务内部再分三层：

- **控制层：**收请求、校验参数、封装响应
- **业务层：**写核心业务逻辑
- **数据访问层：**用MyBatis-Plus操作数据库

(4) 数据层

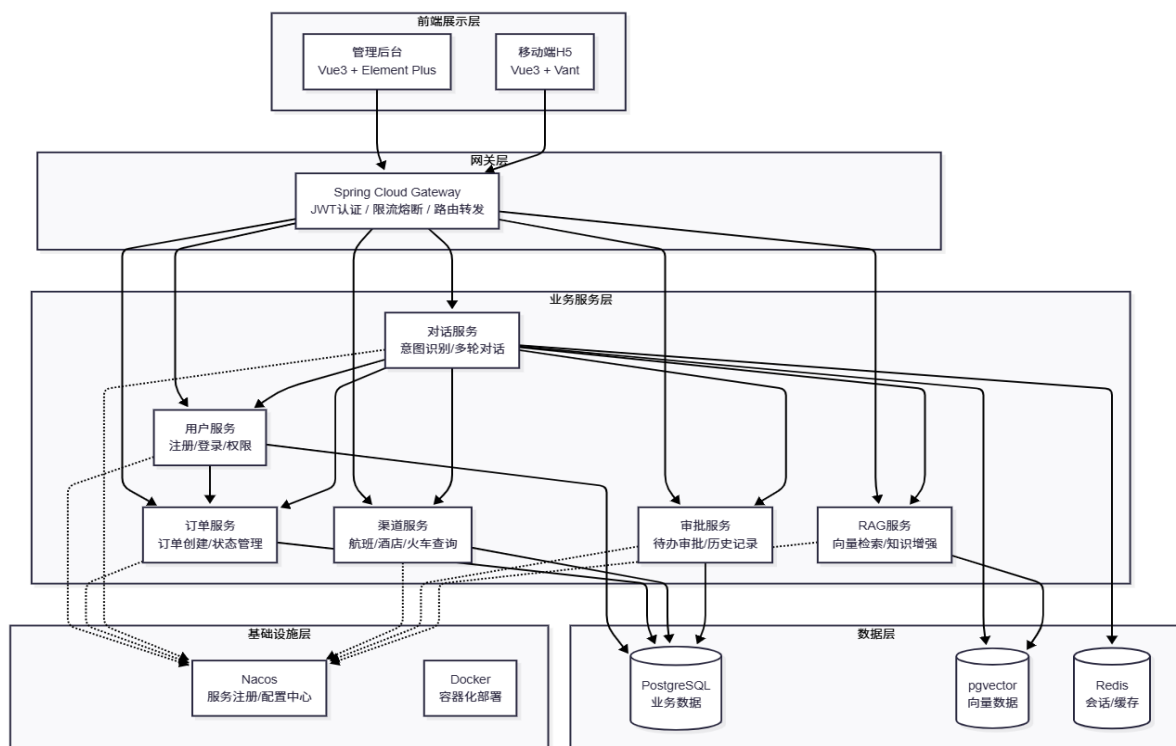
不同类型的数据用不同的存储：

- **业务数据库：**PostgreSQL，存用户、订单、审批等业务数据
- **向量数据库：**pgvector扩展，给RAG用
- **缓存数据库：**Redis，存会话历史、分布式锁、Token缓存

表 4-2 逻辑分层职责说明

层次	职责	技术实现
前端展示层	用户交互界面	Vue 3 + Element Plus（管理后台）/ Vant（移动端）
网关层	请求路由、认证鉴权、限流	Spring Cloud Gateway + Sentinel
控制层	参数校验、响应封装	Spring MVC
业务层	核心业务逻辑	Spring Service
数据访问层	数据库操作	MyBatis-Plus
数据层	数据持久化	PostgreSQL、pgvector、Redis

图 4-1 系统整体架构图



4.1.3 微服务拆分方案

微服务怎么拆？我们按业务边界来拆，不按技术层级。主要这么考虑的：

（1）按业务域拆

用户服务、订单服务、对话服务，分别对应身份认证、交易核心、AI交互这三个业务域。不同业务的规则 and 变化频率不一样，拆开之后各自迭代，互不干扰。

（2）隔离外部依赖

渠道服务专门封装第三方数据源（航班、酒店、火车票接口），把外部变化隔离在单个服务里。第三方接口变了，只改渠道服务就行，其他服务不用动。

（3）控制服务粒度

审批服务是订单服务的伴随服务，单独拆出来，免得订单服务太臃肿。而且审批流程可能独立于订单变化（比如改审批规则），拆开能减少影响范围。

（4）支持独立扩展

对话服务是AI密集型的，对计算资源要求高；订单服务是数据密集型的，对

存储资源要求高。拆开之后，可以按各自的特点独立配资源，成本更优。

按上面这些原则，我们把系统拆成了六个核心微服务。每个服务干啥的，看表4-3。

表 4-3 微服务模块拆分

服务	核心职责	主要功能
用户服务	身份管理与基础信息维护	用户注册登录、JWT认证、权限管理
订单服务	差旅订单全生命周期管理	订单创建、状态变更、查询筛选
渠道服务	外部数据对接与封装	机票/酒店/火车票信息查询
对话服务	自然语言交互与AI集成	通义千问接入、意图识别、工具调用
RAG服务	向量化存储	RAG检索增强
流程审批服务	流程审批管理	待办审批、通过/驳回、历史记录查询

每个服务可独立开发、部署、扩容，服务之间走HTTP协议通信。对话服务是我们这套系统的创新点，它通过Feign调用用户服务和订单服务，把“对话即操作”这个闭环串起来。

4.1.4 服务治理策略

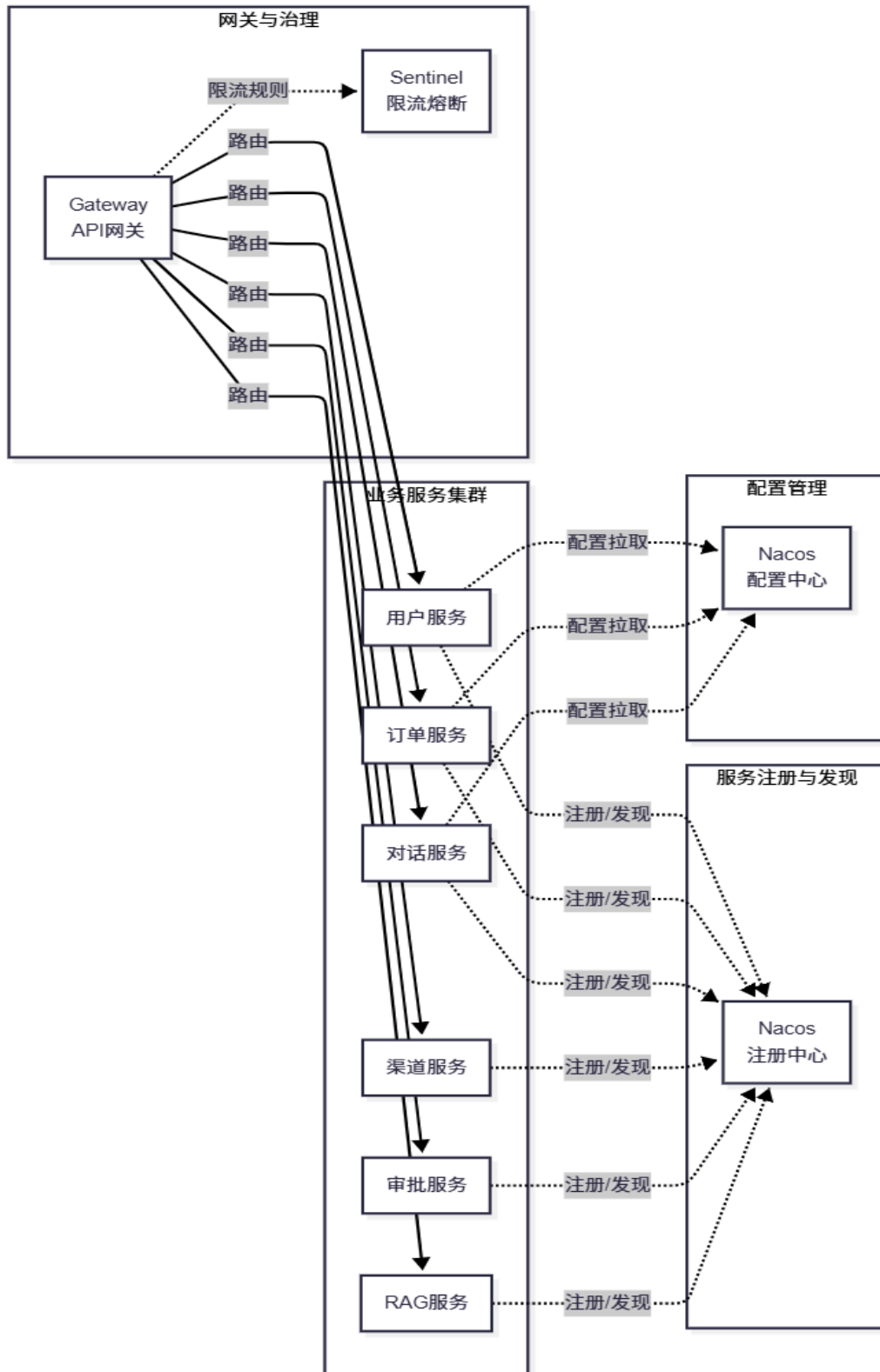
微服务架构虽然灵活，但系统复杂度也上去了，得靠治理策略来保证稳定、好维护。本系统的服务治理情况，可以看表4-4和图4-2。

表 4-4 服务治理策略说明

治理维度	实现组件	策略说明
服务注册与发现	Nacos	自动注册元数据、健康检查、动态服务发现
负载均衡	Spring Cloud LoadBalancer	轮询策略，支持随机、加权响应时间等扩展
熔断降级	Sentinel	错误率/慢调用熔断、半开恢

		复、降级响应
限流	Sentinel	QPS限流、热点参数限流
配置管理	Nacos	多环境隔离（dev/test/prod）、 动态刷新

图 4-2 微服务治理架构图



4.2 数据库设计

本节我们遵循“设计先于实现”原则可以很好地把握好系统边界，掌控项目

实现进度规划。我们按照数据库选型、表设计、ER图展示的顺序来进行讲解。

4.2.1 数据库选型依据

选数据库的时候，我们主要考虑两点：要能支持常规的关系型数据，也要能支持向量存储和检索。几个常用数据库的对比，看表4-5。

表 4-5 数据库选型对比

数据库	适用场景	向量存储支持
MySQL	Web应用、电商系统	MySQL 8.0.31+支持矢量类型，有MyVector插件
SQL Server	Windows生态、.NET应用	SQL Server 2025已支持向量存储
Oracle	大型企业、金融电信	Oracle 23ai及后续版本支持向量检索
PostgreSQL	开源方案、地理空间、AI搜索	支持pgvector、pgvector scale等多种向量扩展插件
OceanBase	高并发、分布式场景	V4.3.3以上支持向量存储检索
KingbaseES	国产化替代、关键行业	V9以上已支持向量存储

综合来看，PostgreSQL最合适——它支持ACID、用起来方便、能做空间位置搜索和关系约束，还能通过pgvector插件搞定向量检索，够轻便。

4.2.2 数据库结构设计说明

- 系统一共设计了 15 张核心表，按业务功能分成四组：
- 组织架构管理：4 张
 - 订单行程管理：5 张
 - 智能对话管理：3 张
 - RAG 知识库管理：3 张
- 下面挨个说明每组表的功能和关系。

4.2.2.1 组织架构管理相关表

组织架构管理这块一共四张表：`company`（企业表）、`departments`（部门表）、`employees`（员工表）、`users`（用户表）。它们一起解决多租户下的数据隔离和权限管理。

`company`（企业表）

企业表是多租户的核心。所有业务数据都靠 `company_id` 关联到具体企业，实现数据隔离。`auto_approve` 字段控制企业要不要走审批流程，方便全局配置。另外，企业有自己的管理账号（`user_name/password`），管理员用这个独立登录后台配置，不用跟员工账号混在一起。

`departments`（部门表）

部门表通过 `parent_id` 自关联，支持树形组织结构（比如“总公司→分公司→部门→小组”这种多级嵌套）。`company_id` 把部门归属到具体企业，支持多企业隔离。

`employees`（员工表）

员工表是连接用户和组织架构的桥梁。`user_id` 关联用户表，支持同一个人不同企业任职（比如一个人同时管多家关联企业的差旅）。`is_primary` 字段用来标记默认企业，解决多企业登录时选哪家的问题。`department_id` 把员工挂到具体部门，方便按部门做数据统计和权限控制。

`users`（用户表）

用户表存登录账户信息，跟员工表是 1:N 的关系（一个用户可以在多个企业任职）。`password` 字段用 BCrypt 加密存，保安全。`roles` 字段简化了角色管理，直接存角色编码（比如 ADMIN、APPROVER、EMPLOYEE），不用搞复杂的角色-权限关联表。

表组关系

`company` → `departments` → `employees` → `users`，这条链就是数据隔离的完整路径。企业 ID 作为全局过滤条件，确保用户只能看到自己所属企业的数据。

4.2.2.2 订单行程管理相关表

订单行程管理一共五张表：`orders`、`journeys`、`tickets`、`passengers`、`approvals`，覆盖差旅订单从下到完成的整个过程。

`orders`（订单表）

订单的核心表，记基本信息和状态。order_no 用时间戳+随机数生成，保证唯一。status 字段管订单状态（DRAFT→ORDERING→PENDING_PAYMENT→PAID），auditStatus 冗余审核状态，方便列表直接查，不用连审批表。order_requirements 存对话里收集的下单参数（JSON），把对话上下文完整传到订单里。approval_id 关联审批记录。

journeys（行程表）

一个订单可以拆成多段行程，支持联程（比如“北京→上海→广州→北京”）。字段设计上兼容多种出行方式：flight_no/train_no/hotel_name/car_no 分别对机票、火车、酒店、用车。departure/arrival/departure_time/arrival_time 记起止点和时间。tickets 字段用 JSON 存票务摘要，方便快速展示。

tickets（票务表）

记每张票的详细信息：票号、乘客信息、价格明细等。passenger_id 关联乘客表，passenger_name 冗余存乘客姓名，免得乘客信息变了订单数据也跟着乱。amount/price/tax_fee/insurance_fee 分别记总价、单价、税费、保险费。has_child/has_baby/has_pet 支持带儿童、婴儿、宠物的需求。

passengers（乘客表）

存常用乘客信息，用户可以维护自己的常用乘客列表，不用每次都重复填。id_type 支持身份证、护照、港澳通行证等。user_id 关联用户，company_id 支持按企业维度管常用乘客（比如企业签约的 VIP 客户）。

approvals（审批表）

记订单审批流程。applicant_id 和 approver_id 都关联员工表，分别是谁申请的、谁批的。trip_items 用 JSON 存出行计划摘要，审批人快速看行程。reason 记出差事由，是审批的重要依据。status 标识审批状态（PENDING/APPROVED/REJECTED）。

表组关系

orders (1) → (N) journeys (1) → (N) tickets，形成订单→行程→票务三层结构。approvals 通过 order_no 关联 orders，审批流程和订单数据解耦。

4.2.2.3 智能对话管理相关表

智能对话管理一共三张表：chat_sessions、chat_messages、prompts，存 AI 对话相关的数据。

chat_sessions（聊天会话表）

记录用户每次和 AI 助手的对话。user_id 和 company_id 关联用户和企业，方便按维度查会话历史。current_stage 记当前对话阶段（比如“信息收集中”“方案推荐中”“订单确认中”），用来断点续传。collected_params 用 JSON 存已经收集到的下单参数，用户中途退出再进来，能恢复状态。last_active_time 用于清理超时会话（24 小时没动静就结束）。

chat_messages（聊天消息表）

记录会话里的每条消息。role 字段标识是谁发的（user/assistant/system），content 存消息正文。metadata 用 JSON 存附加信息（比如推荐方案的原始数据、用户选了什么），方便以后查对话质量。

prompts（提示词表）

支持企业级提示词模板管理。不同商旅场景（机票/酒店/火车）可能需要不同的提示词策略，advisor 字段标识商旅顾问类型，用来做场景化配置。enabled 字段控制是否启用，支持 A/B 测试和灰度发布。company_id 支持企业自定义提示词，user_id 记是谁创建的。

表组关系

chat_sessions (1) → (N) chat_messages，一个会话有多条消息。prompts 独立于会话，作为提示词模板库供对话服务调用。

4.2.2.4 RAG 知识库管理相关表

RAG 知识库管理一共三张表：rag_files、policy_documents、standards，支撑企业差旅政策的知识库和检索增强。

rag_files（RAG 文件表）

记录企业上传的原始政策文件。file_path 存文件路径（本地或 OSS），file_size 和 mime_type 记文件元信息。scene 字段标识文件用途（比如 POLICY_2025 、 TRAVEL_RULE ）。 status 字段记文件处理状态（UPLOADED→PARSING→VECTORIZED→FAILED），方便异步处理时追踪。这张表只做记录备查，不能删也不能改，保证知识变更可追溯。

policy_documents（政策信息表）

这是 RAG 向量检索的核心表。content 字段存分块后的政策内容（每块约 500 字符），embedding 字段存通义千问 Embedding 接口生成的 1536 维向量，支持余弦相似度检索。company_id 和 dept_id 做数据隔离，支持企业级和部门级政策。metadata 存原始文件信息（文件名、上传时间、版本号等），方便检索结果溯源。

standards（差标表）

存企业差旅标准的量化规则。content 字段用结构化格式存差标配置，比如各出行方式（机票、酒店、火车）不同等级对应的金额上限。

差标表和政策信息表是互补的：政策信息表用来做自然语言问答（比如“公司的住宿标准是多少？”），差标表用来做程序化合规校验（比如订单金额有没有超限）。

表组关系

rag_files 记录原始文件上传记录，policy_documents 存向量化后的政策内容，两者通过文件名和上传时间关联。standards 独立于 RAG 流程，作为结构化规则库供订单合规校验调用。

4.2.2.5 表组关系总结

表4-6 表分组关系

分组表	核心表	辅助表	关联关系
组织架构管理	company	Departments 、 employees、 users	company 作为数据隔离根节点
订单行程管理	orders	journeys、 tickets、 passenge、 approvals	orders 为业务核心，1:N 关联行程和审批
智能对话管理	chat_sessions	chat_messages 、 prompts	会话与消息 1:N，提示词独立调用
RAG 知识库管理	policy_documents	rag_files 、 standards	政策文档为检索核心，差标为规则补充

这15张表的设计主要遵循几点原则：

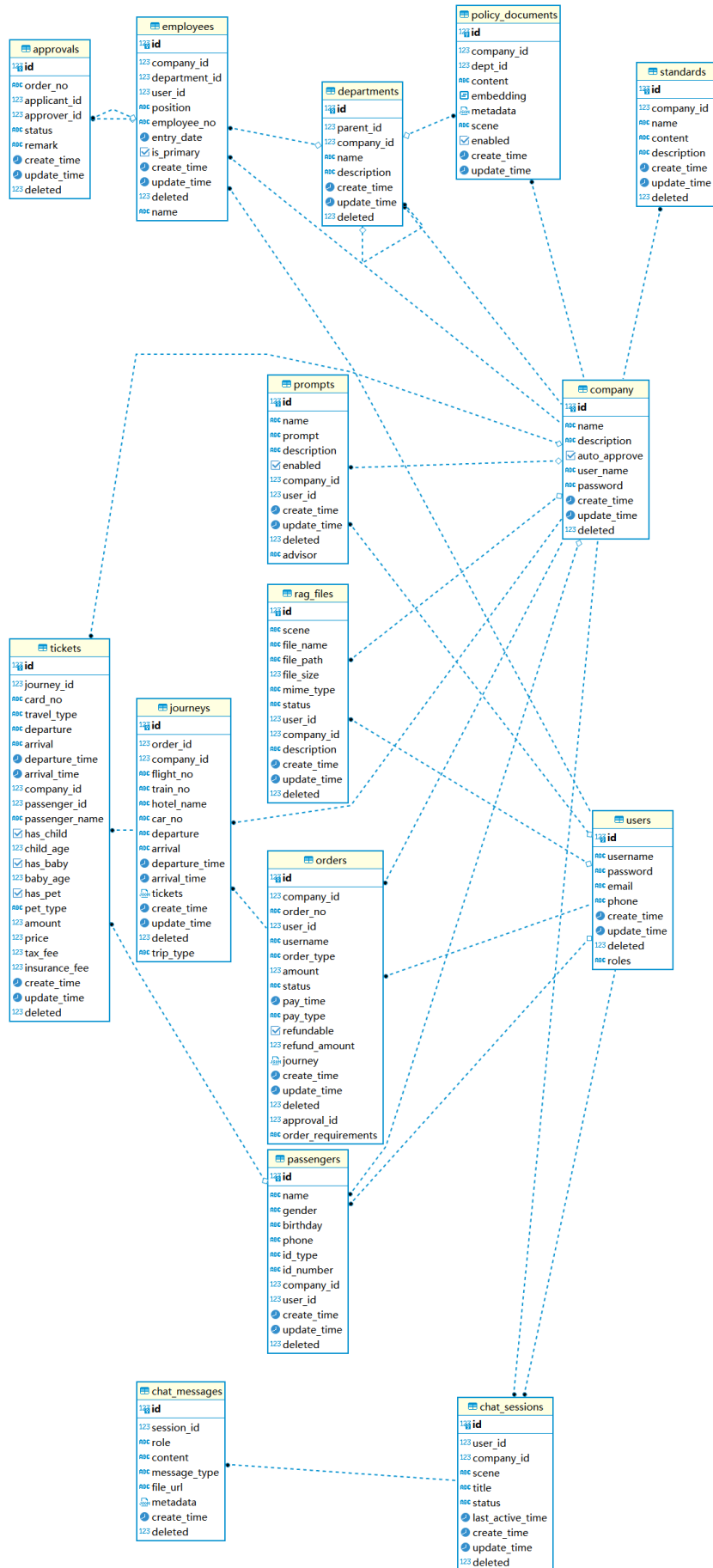
- **数据隔离**：用company_id做隔离，支持多租户
- **逻辑删除**：用deleted字段标记删除，不真删，方便追溯
- **时间统一**：时间字段统一用CURRENT_TIMESTAMP，让数据库自己维护
- **冗余字段**：关键业务表加一些冗余字段（比如orders表存username），少做关联查询，提升性能

4.2.3 业务数据库 ER 图

4.2.3.1 ER 关系图

前面说了这么多，来看一下数据库的ER图。

图 4-3 ER 实体关系图



4.2.4 数据库表结构设计

数据库表汇总：

序号	表名	中文名	备注
1	users	用户表	系统登录基础表
2	company	企业表	企业主体表
3	departments	部门表	支持树形部门结构
4	employees	员工表	关联用户与组织架构
5	orders	订单表	核心业务订单数据
6	journeys	行程段表	支持多段行程拆分
7	tickets	票务明细表	每张票务详细记录
8	passengers	乘客表	常用乘客信息维护
9	approvals	审批表	支持审批流程
10	standards	差标表	差标规则定义表
11	prompts	提示词表	提示词模板管理
12	chat_sessions	聊天会话表	聊天会话表
13	chat_messages	聊天消息表	绑定会话的聊天消息表
14	rag_files	RAG 文件表	方便企业和部门核查上传记录
15	policy_documents	政策信息表	用户向量存储和检索

下面我们来看看数据库表的详细结构：

4.2.4.1 company (企业表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	主键
name	VARCHAR(128)	NOT NULL	-	企业名称
description	TEXT	-	-	描述
auto_approve	BOOLEAN	NOT NULL	FALSE	自动审批
user_name	VARCHAR(100)	UK, NOT NULL	-	企业账号(唯一)
password	VARCHAR(255)	NOT NULL	-	密码(加密串)

字段	类型	约束	默认值	说明
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.2 departments (部门表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	主键
parent_id	BIGINT	FK	NULL	父部门 ID(自关联)
company_id	BIGINT	FK, NOT NULL	-	企业 ID
name	VARCHAR(128)	NOT NULL	-	部门名称
description	TEXT	-	-	描述
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.3 employees (员工表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	主键
company_id	BIGINT	FK, NOT NULL	-	企业 ID

字段	类型	约束	默认值	说明
department_id	BIGINT	FK	–	部门 ID
user_id	BIGINT	UK, FK, NOT NULL	–	用户 ID (唯一)
name	VARCHAR(100)	NOT NULL	–	姓名
position	VARCHAR(64)	–	–	职位
employee_no	VARCHAR(64)	–	–	工号
entry_date	DATE	–	NULL	入职日期
is_primary	BOOLEAN	NOT NULL	FALSE	是否默认企业
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.4 users (用户表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
username	VARCHAR(64)	UK, NOT NULL	–	用户名 (唯一)
password	VARCHAR(255)	NOT NULL	–	密码 (加密串)
email	VARCHAR(128)	–	–	邮箱
phone	VARCHAR(32)	–	–	手机
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

字段	类型	约束	默认值	说明
roles	VARCHAR(255)	NOT NULL	–	角色（多个逗号分隔）

4.2.4.5 orders (订单表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
company_id	BIGINT	FK, NOT NULL	–	企业 ID
order_no	VARCHAR(64)	UK, NOT NULL	–	订单号（唯一）
user_id	BIGINT	FK, NOT NULL	–	用户 ID
username	VARCHAR(64)	NOT NULL	–	用户名（冗余）
order_type	VARCHAR(32)	NOT NULL	–	订单类型
amount	DECIMAL(12,2)	NOT NULL	–	金额
status	VARCHAR(32)	NOT NULL	DRAFT	订单状态
auditStatus	VARCHAR(32)	NOT NULL	PENDING	审核状态
pay_time	TIMESTAMP	–	NULL	支付时间
pay_type	VARCHAR(32)	–	NULL	支付类型
refundable	BOOLEAN	–	TRUE	是否可退改
refund_amount	DECIMAL(12,2)	–	0.00	退款金额
journey	JSONB	–	–	行程信息（JSON）
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间

字段	类型	约束	默认值	说明
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除
order_requirements	TEXT	NOT NULL	-	订单参数要求 JSON 字符串：该参数是对话上下文收集到的下单参数
approval_id	BIGINT	-	-	申请单 ID，用于流程审批

4.2.4.6 journeys (行程表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	主键
order_id	BIGINT	FK, NOT NULL	-	订单 ID
company_id	BIGINT	FK, NOT NULL	-	企业 ID
flight_no	VARCHAR(32)	-	-	航班号
train_no	VARCHAR(32)	-	-	火车车次
hotel_name	VARCHAR(128)	-	-	酒店名称
car_no	VARCHAR(32)	-	-	车牌号
departure	VARCHAR(128)	NOT NULL	-	出发地
arrival	VARCHAR(128)	NOT NULL	-	目的地
departure_time	TIMESTAMP	NOT NULL	-	出发时间

字段	类型	约束	默认值	说明
arrival_time	TIMESTAMP	NOT NULL	–	到达时间
tickets	JSONB	–	–	票务信息（JSON）
trip_type	VARCHAR(50)	NOT NULL	–	行程类型
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.7 tickets (票务表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
journey_id	BIGINT	FK, NOT NULL	–	行程 ID
card_no	VARCHAR(64)	–	–	票号
travel_type	VARCHAR(32)	–	–	票务类型
departure	VARCHAR(128)	NOT NULL	–	出发地
arrival	VARCHAR(128)	NOT NULL	–	目的地
departure_time	TIMESTAMP	NOT NULL	–	出发时间
arrival_time	TIMESTAMP	NOT NULL	–	到达时间
company_id	BIGINT	FK, NOT NULL	–	企业 ID
passenger_id	BIGINT	FK, NOT NULL	–	乘客 ID
passenger_name	VARCHAR(64)	NOT NULL	–	乘客姓名

字段	类型	约束	默认值	说明
has_child	BOOLEAN	NOT NULL	FALSE	是否有儿童
child_age	INTEGER	NOT NULL	0	儿童年龄
has_baby	BOOLEAN	NOT NULL	FALSE	是否有婴儿
baby_age	INTEGER	NOT NULL	0	婴儿年龄
has_pet	BOOLEAN	NOT NULL	FALSE	是否有宠物
pet_type	VARCHAR(32)	–	–	宠物类型
amount	DECIMAL(12,2)	NOT NULL	–	总价
price	DECIMAL(12,2)	–	0.00	单价
tax_fee	DECIMAL(12,2)	–	0.00	税费
insurance_fee	DECIMAL(12,2)	–	0.00	保险费
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.8. passengers (乘客表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
name	VARCHAR(64)	NOT NULL	–	姓名
gender	VARCHAR(8)	–	–	性别
birthday	VARCHAR(20)	–	–	生日

字段	类型	约束	默认值	说明
phone	VARCHAR(32)	–	–	手机
id_type	VARCHAR(32)	NOT NULL	ID_CARD	证件类型
id_number	VARCHAR(64)	NOT NULL	–	证件号码
company_id	BIGINT	FK	–	企业 ID
user_id	BIGINT	FK	–	用户 ID
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.9 approvals (审批表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
order_no	VARCHAR(64)	UK, NOT NULL	–	订单号（唯一）
applicant_id	BIGINT	FK, NOT NULL	–	申请人 ID(员工 ID)
approver_id	BIGINT	FK, NOT NULL	–	审批人 ID（员工 ID）
trip_items	JSONB	NOT NULL	–	出行列表：用于用户发起订单出行的多端行程记录，若是先审批那么可以约定行程规范

字段	类型	约束	默认值	说明
reason	VARCHAR(255)	NOT NULL	–	出差事由，写明出差的原因
status	VARCHAR(32)	NOT NULL	–	审批状态
remark	TEXT	–	–	审批意见
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.10. standards (差标表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
company_id	BIGINT	FK, NOT NULL	–	企业 ID
name	VARCHAR(128)	NOT NULL	–	标准名称
content	TEXT	NOT NULL	–	标准内容 (JSON)
description	TEXT	–	–	描述
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.11. prompts (提示词表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
name	VARCHAR(128)	NOT NULL	–	提示词名称
prompt	TEXT	NOT NULL	–	提示词内容
advisor	VARCHAR(50)	NOT NULL	–	商旅顾问类型
description	TEXT	–	–	描述
enabled	BOOLEAN	NOT NULL	TRUE	是否启用
company_id	BIGINT	FK, NOT NULL	–	企业 ID
user_id	BIGINT	FK, NOT NULL	–	用户 ID
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.12. chat_sessions (聊天会话表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
user_id	BIGINT	FK, NOT NULL	–	用户 ID
company_id	BIGINT	FK, NOT NULL	–	企业 ID

字段	类型	约束	默认值	说明
scene	VARCHAR(20)	NOT NULL	–	场景
title	VARCHAR(255)	NOT NULL	–	会话标题
status	VARCHAR(20)	NOT NULL	–	会话状态
current_stage	VARCHAR(50)	NOT NULL	–	当前对话阶段
collected_params	TEXT	–	–	已收集参数
last_active_time	TIMESTAMP	NOT NULL	–	最后活跃时间
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.13. chat_messages (聊天消息表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键
session_id	BIGINT	FK, NOT NULL	–	会话 ID
role	VARCHAR(20)	NOT NULL	–	角色
content	TEXT	NOT NULL	–	消息内容
message_type	VARCHAR(32)	–	–	消息类型
file_url	VARCHAR(512)	–	–	文件地址

字段	类型	约束	默认值	说明
metadata	JSONB	-	-	元数据
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
deleted	INTEGER	NOT NULL	0	逻辑删除（0-未删除，1-已删除）

4.2.4.14. rag_files (RAG 文件表)

字段	类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	-	主键
scene	VARCHAR(50)	NOT NULL	-	场景
file_name	VARCHAR(255)	NOT NULL	-	文件名
file_path	VARCHAR(512)	NOT NULL	-	文件路径
file_size	BIGINT	NOT NULL	-	文件大小
mime_type	VARCHAR(128)	NOT NULL	-	MIME 类型
status	VARCHAR(32)	-	-	状态
user_id	BIGINT	FK	-	用户 ID
company_id	BIGINT	FK	-	企业 ID
description	TEXT	-	-	描述
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间
deleted	INTEGER	NOT NULL	0	逻辑删除

4.2.4.15. policy_documents (政策信息表)

字段名	数据类型	约束	默认值	说明
id	BIGINT	PK, AUTO_INCREMENT	–	主键 ID (自增)
company_id	BIGINT	FK, NOT NULL	–	企业 ID (必填, 数据隔离)
dept_id	BIGINT	FK	NULL	部门 ID (可选, NULL 表示公司级通用政策)
content	TEXT	NOT NULL	–	政策文档内容
embedding	VECTOR(1536)	NOT NULL	–	向量数据 (用于相似度检索)
metadata	JSONB	NOT NULL	–	元数据 (文件名、大小、上传时间等)
scene	VARCHAR(50)	NOT NULL	–	政策场景 (TRAIN/FLIGHT/HOTEL/CAR/QA 等)
enabled	BOOLEAN	NOT NULL	TRUE	是否启用 (软删除/禁用)
create_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	创建时间
update_time	TIMESTAMP	NOT NULL	CURRENT_TIMESTAMP	更新时间

4.2.5 向量数据库扩展说明

文档切片、向量化存储和检索，都可以通过PostgreSQL的pgvector扩展来实现。

使用pgvector有几个前提：

- PostgreSQL版本要够新
- 创建数据库实例后开启pgvector支持
- 添加必要的配置

4.2.5.1 配置参数说明

下面说说pgvector的几个关键配置参数，以及为什么这么选。

表4-6 向量核心配置参数选型

参数	选值	选型依据
index-type	HNSW	HNSW（分层可导航小世界图）索引相比 IVFFlat 具有更高的查询精度和更快的检索速度，适合实时问答场景
distance-type	COSINE_DISTANCE	余弦距离衡量向量方向相似度，对文本语义检索效果最佳，与通义千问 Embedding 模型的训练目标一致
dimensions	1536	通义千问 text-embedding-v3 模型输出向量维度为 1536，需与此保持一致

4.2.5.2 向量检索的原理

向量检索的过程：

- 先算查询向量和存储向量之间的余弦距离[55]
- 按相似度从高到低排序，返回最相关的文档片段
- 余弦距离范围是0到2，值越小越相似。我们设了0.7的阈值，只返回距离小于0.7的结果，这样政策匹配更准。

4.2.5.3 向量化扩展配置

要开启向量化，进 PostgreSQL 数据库，执行这条命令就行：

```
CREATE EXTENSION IF NOT EXISTS vector;
```

接下来在 Spring Boot 项目里配一下 AI 集成向量，代码长这样：

```
spring:
  # Spring AI 通义千问配置
  ai:
    dashscope:
      # 通义千问 API Key
```

```

api-key: sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
chat:
  options:
    # 可切换 LLM qwen-max、qwen-plus
    model: qwen-turbo
embedding:
  options:
    # M6.1 推荐 v3 (1536 维)
    model: text-embedding-v3
vectorstore:
  pgvector:
    index-type: HNSW          # 使用 HNSW 索引, 查询性能好
    distance-type: COSINE_DISTANCE # 余弦距离
    dimensions: 1536          # 通义千问 embedding 维度
    initialize-schema: false #首次运行时自动创建表结构
    remove-existing-vector-store-table: false # 禁止设为 true
    schema-name: public
    table-name: policy_documents # 自定义表名

```

向量检索的原理,就是通过计算向量的距离来判断信息匹配度——意思相近的词,向量距离也近。而文档切片、存储、向量化这些,正是 RAG 增强检索的基础。

4.3 核心模块划分

按前面说的微服务拆分方案,系统分成了六个核心模块,各管各的,边界清楚。

4.3.1 用户服务模块

用户服务模块管用户身份和基础信息,是系统的认证授权中心。主要功能有:

- **注册与登录:** 用户用账号密码登录,系统生成JWT令牌
- **信息管理:** 查改用户基本信息,包括重置密码
- **权限管理:** 给用户分配角色(普通员工、审批人、管理员),按角色控制访问权限
- **企业信息管理:** 查改企业信息
- **部门与员工管理:** 维护企业组织架构

4.3.2 订单服务模块

订单服务模块管差旅订单从创建到完成的整个过程，是系统的核心业务模块。主要功能有：

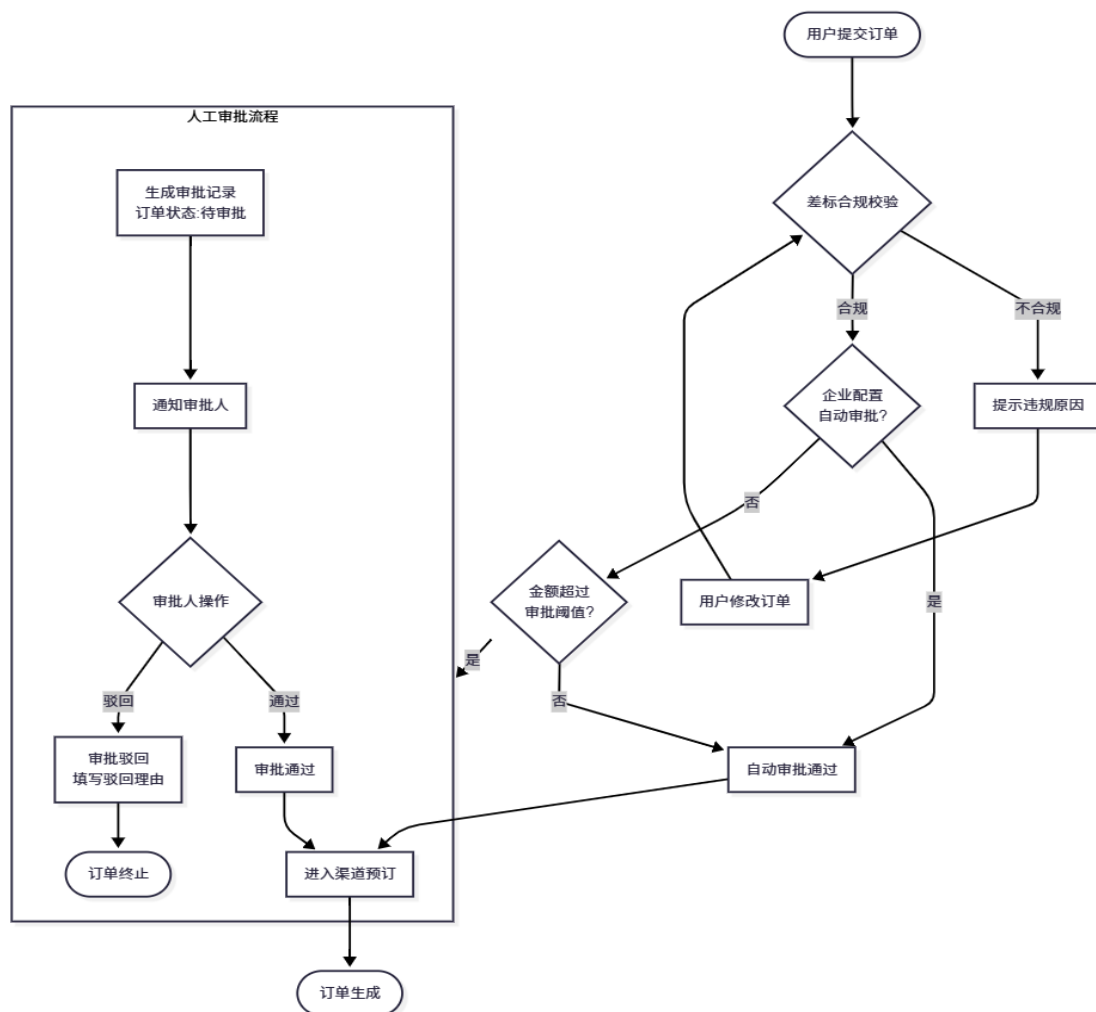
- **订单创建：**从对话服务接收结构化订单数据，生成正式订单
- **订单状态管理：**控制订单状态流转（待审批→已审批→已预订→已完成/已取消）
- **订单查询：**支持按用户、时间、状态等条件组合查询
- **订单详情展示：**展示完整订单信息，包括行程明细、费用明细、审批记录

4.3.3 流程审批模块

流程审批模块管差旅申请的审批流程。主要功能有：

- **待办审批列表：**展示当前用户待批的申请
- **审批操作：**支持通过或驳回，驳回时填理由
- **历史审批记录：**可查历史审批记录，方便追溯
- **审批规则配置：**支持配置自动审批等规则

图 4-4 订单审批流程



4.3.4 渠道服务模块

渠道服务模块负责对接和封装外部数据。主要功能有：

- 航班信息查询：接第三方航班API，查航班信息
- 酒店信息查询：查酒店信息
- 火车票信息查询：查火车票信息
- 数据缓存：高频查询结果做缓存，响应更快

4.3.5 对话服务模块

对话服务模块是系统的核心创新点，负责实现自然语言对话交互。主要功能有：

- 接入和封装通义千问API
- 基于提示词工程做意图识别和槽位提取
- 基于Redis管理多轮对话状态
- 通过Tool Calling调用业务操作
- 生成对话回复，做异常引导

4.3.6 RAG 服务模块

RAG服务模块是系统的核心支撑，负责企业政策知识库的向量化存储和检索增强。主要功能有：

- 文档上传解析和文本分块
- 调用Embedding接口生成向量
- pgvector向量存储和相似度检索
- 把检索结果喂给大模型，做知识增强问答

4.4 本章小结

这一章主要做总体设计，分三块：

（1）架构设计

- 技术栈：Spring Cloud Alibaba + Vue + 通义千问
- 四层架构：前端展示层、网关层、业务服务层、数据层
- 微服务拆分：用户、订单、渠道、对话、审批、RAG
- 服务治理：服务注册发现、负载均衡、熔断降级等

（2）数据库设计

- 存储方案：PostgreSQL + pgvector
- 表结构：15 张核心表，ER 图，详细表设计
- 向量库：配置说明

（3）模块划分

- 六个核心模块，职责边界清楚

这些设计给后面的详细实现打了个底。

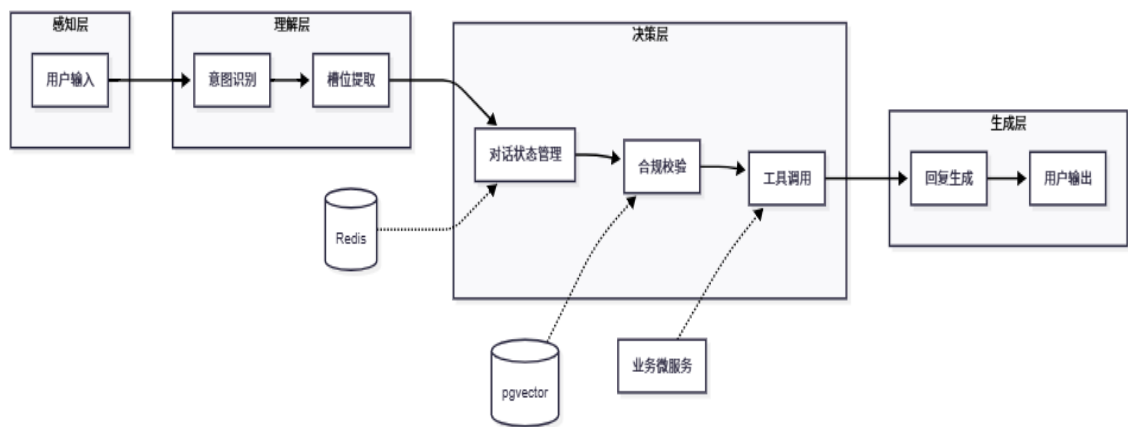
5 系统详细设计与实现

第 4 章说了，系统按微服务拆成了六个核心模块：用户、订单、对话、渠道、审批、RAG。对话服务是这套系统的创新点，所以这一章重点讲它怎么设计和实现，其他模块只简单提一下。

5.1 对话服务核心实现

对话服务负责实现自然语言对话交互，是连接用户和业务系统的智能中枢。它的处理流程分四个阶段：感知→理解→决策→生成。如图 5-1 所示，用户输入先过 API 接入层，然后依次走自然语言理解、多轮对话管理、服务调用，最后生成回复返回给用户。

图 5-1 对话服务处理流程架构图



下面挨个说说每个模块的设计和实现。

5.1.1 通义千问 API 接入与封装

5.1.1.1 设计思路

系统里 Java 调用通义千问 API，是 AI 能力的核心来源。API 接入层主要解决三个问题：

- **稳定性：**网络波动或 API 限流时，系统要有容错机制，不能因为 API 挂了导致整个对话服务崩掉
- **可维护性：**API 密钥、模型版本这些配置要支持动态刷新，改完不用重启服务

- **兼容性：**通义千问 API 版本可能会更新，响应格式要做兼容处理

针对上面几点，API 接入层这么设计的：

- **超时分离：**连接超时设 10 秒，读取超时设 30 秒。连接超时应对网络不通，读取超时应对模型推理慢的情况
- **降级策略：**API 调用失败时，返回预设的友好提示，不直接抛异常
- **配置外部化：**API 密钥、模型版本放 Nacos 配置中心管理，支持动态刷新

5.1.1.2 核心实现

API 客户端这块，把调用通义千问的 HTTP 逻辑封装了一下。核心代码长这样：

```
@Component
public class TongyiQianwenClient {

    @Autowired
    private TongyiConfig config;
    private RestTemplate restTemplate;

    @PostConstruct
    public void init() {
        SimpleClientHttpRequestFactory factory = new
SimpleClientHttpRequestFactory();
        factory.setConnectTimeout(10000);    // 连接超时10秒
        factory.setReadTimeout(30000);      // 读取超时30秒
        this.restTemplate = new RestTemplate(factory);
    }

    public String chat(ChatRequest request) {
        try {
            // 构建请求
            HttpEntity<Map<String, Object>> entity =
buildRequest(request);
            // 调用API
```

```

        ResponseEntity<Map> response = restTemplate.postForEntity(
            config.getApiUrl(), entity, Map.class);
        // 解析响应（兼容多种格式）
        return extractContent(response.getBody());
    } catch (RestClientException e) {
        log.error("通义千问API调用失败", e);
        return "AI服务暂时不可用，请稍后重试或联系人工客服。";
    }
}
}

```

关键设计说明：几个关键设计点：

- **超时分离**：connectTimeout和readTimeout分开设，免得网络一抖就卡死
- **失败降级**：调用失败时返回友好提示，不抛异常，保证对话流程不中断
- **格式兼容**：chextractContent方法同时支持output.text和choices[].message.content两种格式，应对API版本变化

5.1.2 自然语言理解实现

自然语言理解得结合用户上下文才行。我们通过提示词引导用户输入专业领域的知识问答，然后提取槽位参数来补全结构化参数。同时配合意图识别和问题澄清，最终把参数补全。

5.1.2.1 提示词模板设计

自然语言理解这块，核心就是提示词模板。我们设计提示词的时候，主要遵循三个原则：

原则	说明	在本系统中的应用
角色设定	明确AI的身份和职责边界	“你是专业的商旅助手，帮助用户完成差旅预订”
任务分解	将复杂任务拆解为可执行的子任务	先识别意图，再提取槽位，最后生成订单
输出约束	限定输出格式，便于程序解析	强制要求输出JSON格式，并给出Schema

系统里主要用了两个提示词模板：

(1) 意图识别提示词

用来判断用户想干什么（预订、咨询、闲聊），输出JSON格式的分类结果。用了few-shot的方法(给模型几个例子)，帮它理解分类标准。

(2) 订单生成提示词

用来从对话里提取结构化的订单信息。模板里带了差标规则，用户要是超标了，就返回违规提示。

设计上的权衡

提示词不能太短，短了约束不够，模型输出不稳；也不能太长，长了费token、响应慢。调了好几版，最后定在800-1200字符之间。

5.1.2.2 意图识别与槽位提取

意图识别和槽位提取是自然语言理解的两个核心任务。我们用了“提示词引导+规则补充”的混合策略：

- **主策略：**用提示词让大模型直接输出意图分类和槽位信息
- **补充策略：**对模型输出做后处理，比如把“明天”转成具体日期，把“成都东站”“成都双流机场”都映射成“成都”

槽位提取的核心逻辑如下（精简版）：

```
public Map<String, Object> extractSlots(String userInput,
Map<String, Object> currentSlots) {
    // 构建带上下文的提示词
    String prompt = buildPromptWithContext(userInput,
currentSlots);

    // 调用大模型
    ChatRequest request = new ChatRequest();
    request.addSystemMessage(PromptTemplate.SYSTEM_PROMPT);
    request.addUserMessage(prompt);
    String response = tongyiClient.chat(request);

    // 解析JSON响应
    JSONObject result = JSON.parseObject(extractJson(response));
```

```
// 合并到现有槽位
return mergeSlots(currentSlots,
result.getJSONObject("order"));
}
```

- **槽位补全策略：**槽位怎么补？发现信息缺了，就按缺什么问什么。比如缺“出发地”，就问“请问您从哪个城市出发？”，不会只丢一句“请补充信息”。

5.1.2.4 多意图处理机制

用户有时候一句话里带好几个意图，比如“帮我订一张机票，顺便查一下差旅标准”。多意图的处理，我们用了“优先级队列+分派”的策略：

优先级	意图类型	处理方式
1（最高）	预订类	立即执行，其他意图排队
2	订单类	预订完成后执行
3	政策咨询类	异步返回结果，不阻塞主流程

设计依据：预订是用户的核心诉求，应优先处理；政策咨询可在预订过程中穿插回答，不影响主流程。

5.1.3 多轮对话管理实现

5.1.3.1 会话状态管理设计

多轮对话得把会话状态记下来，包括已经收集到的参数、当前对话阶段、聊天历史等。我们用了 Redis 来存，主要设计思路如下：

设计要点	说明
存储结构	Hash结构，key为 dialog:session:{sessionId}，field 存储各状态字段
过期策略	24小时TTL，无活动自动清理
断点续传	用户中途退出后重新进入，可从上次中

	断处继续
--	------

为什么用Redis，不用内存存储？

- 对话服务可能会部署多个实例，用内存存的话，不同实例之间没法共享会话状态
- Redis支持持久化，服务重启了会话还在
- 设了24小时TTL自动清理，不会内存泄漏

5.1.3.2 差标合规校验集成

合规性校验不是在订单提交后才做，而是在对话过程中实时执行。流程如下：

1. 用户选好出行方案后，系统调用合规校验服务
2. 校验服务去查企业的差标配置，比对订单金额和舱位等级
3. 校验通过 → 继续走订单生成流程
4. 校验不通过 → 提示违规原因，引导用户调整方案

这么设计的好处是，把合规管控从“事后审核”变成了“事中拦截”，用户不会等到填完一堆信息才被告知超标。

5.1.4 服务调用实现

5.1.4.1 工具调用机制设计

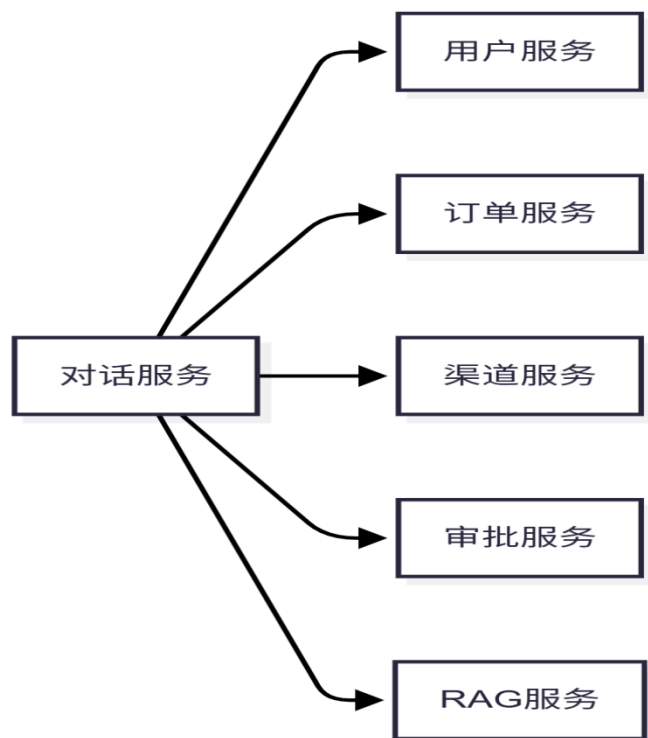
我们用通义千问的Function Calling能力，让AI能主动调用业务接口。工具调用这块，主要按三个原则来设计：

- **声明式定义：**每个工具（比如“查询航班”“创建订单”）用JSON Schema格式声明需要哪些输入参数
- **透明调用：**AI自己决定什么时候调用、传什么参数，系统只管执行
- **结果反馈：**工具执行完的结果返回给AI，AI再根据结果生成回复

5.1.4.2 下游微服务集成

对话服务通过Feign客户端调用其他微服务，调用关系见图5-2。

图5-2 对话服务调用关系图



Feign客户端配置，精简版如下：

```

@FeignClient(name = "travel-order", path = "/order")
public interface OrderFeignClient {
    @PostMapping("/create")
    Response<Order> createOrder(@RequestBody OrderCreateDTO dto);
}
  
```

5.1.5 对话回复生成实现

5.1.5.1 回复模板设计

回复生成这块，用了“模板+动态填充”的方式。不同场景用不同的模板：

场景	模板示例	填充内容
订单确认	“已为您生成订单： {订单类型}，{出发地}→ {目的地}，{出发日期}”	订单信息
信息缺失	“请问您的{缺失字段} 段}是？”	缺失字段名
违规提示	“订单不符合差标： {违规原因}。请调整后重	违规原因

	试。”	
--	-----	--

设计上，回复要简洁、明确、能落地。别用“系统处理中”“请稍后”这种模糊的说法。

5.1.5.2 异常引导策略

AI要是听不懂用户的意思或者处理失败了，系统会分层引导：

- 1. 先提示用户重新说一遍，比如“抱歉，我没理解您的意思，请再说一遍？”
- 2. 还不行的话，给个示例引导，比如“您可以试试这样说：‘帮我订明天北京到上海的机票’”
- 3. 实在不行，转人工客服（预留了接口）

5.1.6 RAG 知识增强实现

5.1.6.1 向量库构建流程

RAG知识增强分两块：离线构建和在线检索。离线构建的流程见图5-3：

图5-3 向量库构建流程图



几个关键设计：

- **分块大小**：每块500字符，块之间重叠50字符。块太小语义不完整，太大检索精度会下降
- **向量维度**：1536维，跟通义千问text-embedding-v3模型的输出维度保持一致
- **索引类型**：用HNSW索引，比IVFFlat查询精度更高、速度更快，适合实时问答场景

5.1.6.2 相似度检索与知识注入

在线检索时，系统把用户的问题也转成向量，然后在pgvector里算余弦相似度，返回最相关的3-5个政策片段。

为什么取3-5条？这是试出来的经验值。太少（1-2条）可能漏掉关键政策，

太多（6条以上）会带进来一堆没用的信息，还费token。

检索结果作为背景信息喂给大模型，提示词的格式如下：

根据以下企业政策回答用户问题：

[政策片段1]

[政策片段2]

[政策片段3]

用户问题：{用户输入}

5.2 业务微服务实现要点

除了对话服务，其他几个微服务的实现都比较常规，下面简单说一下核心点。

5.2.1 用户服务

用户服务采用Spring Security + JWT实现认证授权。密码采用BCrypt加密存储，Token有效期设为24小时，支持Token刷新机制。

5.2.2 订单服务

订单服务用MyBatis-Plus做ORM。订单号生成规则是：ORD + yyyyMMddHHmmss + 4位随机数，保证全局唯一。订单状态流转用状态机模式来实现，确保状态变更合法。

5.2.3 审批服务

审批服务提供待办审批列表、通过/驳回、历史记录查询这几个功能。审批人只能批自己部门及下属部门的订单，权限控制在服务层做。

5.2.4 渠道服务

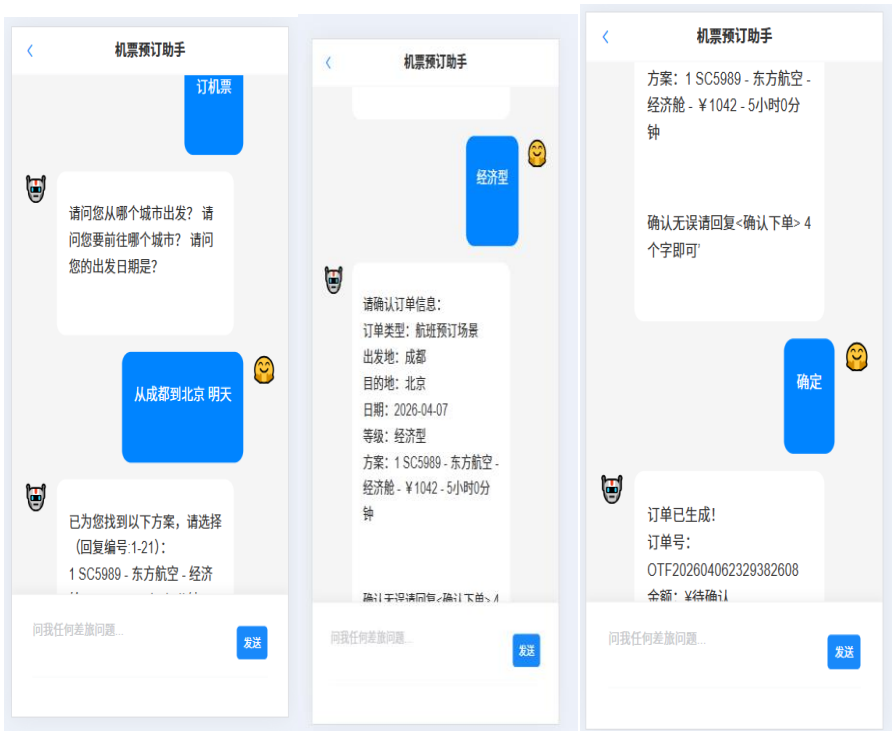
渠道服务封装第三方数据源。为降低开发复杂度，当前版本采用Mock数据实现，预留了对接真实API的接口。高频查询结果缓存至Redis，缓存有效期30分钟。

5.3 系统界面展示

5.3.1 对话下单界面

用户选好前端引导场景进入聊天下单，参数就会进入槽位。比如用户说“订机票”“从成都到北京 明天”，系统就会推荐方案。

图 5-4 对话下单界面截图



界面分三个区域：顶部是状态栏，显示当前会话信息；中间是对话历史区，展示用户和 AI 的聊天记录；底部是输入区，支持打字发送。用户第一次进来时，系统会自动显示欢迎语和使用引导。

5.3.2 订单管理界面

订单管理界面可以查订单列表，也能看订单详情。

图 5-5 订单管理界面截图

智能商旅

数据看板

系统管理

出行管理

订单管理

差旅管理

流程审批

智能管理

智能商旅后台管理系统

订单管理

订单号

订单类型

订单状态

审核状态

新增

重置

ID	企业名称	订单号	类型	金额	订单状态	审核状态	用户名	创建时间	操作
46	宇富公司	OTF202604032247019808	机票	¥ 1186.00	待支付	通过	universe	2026-04-03 22:47:01	详情
45	宇富公司	OTF20260403174755D6FD	机票	¥ 1128.00	待支付	通过	universe	2026-04-03 17:47:55	详情
44	宇富公司	OTF20260403174637EC87	机票	¥ 1074.00	待支付	通过	universe	2026-04-03 17:46:37	详情
43	宇富公司	OTF2026040317294462F9	机票	¥ 1005.00	草稿	通过	universe	2026-04-03 17:29:44	详情
42	宇富公司	OTF202604031706175502	机票	¥ 1347.00	待支付	通过	universe	2026-04-03 17:06:17	详情
41	宇富公司	OTF202604031704373F08	机票	¥ 1367.00	草稿	拒绝	universe	2026-04-03 17:04:37	详情
40	宇富公司	OTF202604031639184C1D	机票	¥ 1097.00	待支付	通过	universe	2026-04-03 16:39:18	详情
38	宇富公司	OTF202604031622475888	机票	¥ 1114.00	待支付	通过	universe	2026-04-03 16:22:47	详情
39	宇富公司	OTF202604031634508436	机票	¥ 1230.00	待支付	通过	universe	2026-04-03 16:34:50	详情
33	宇富公司	OTF20260402195834F7CD	机票	¥ 1283.00	待支付	通过	universe	2026-04-02 19:58:41	详情

1

2

10 / page

界面可以按订单状态、类型等条件组合查询。订单列表按创建时间倒着排，每条显示订单号、类型、金额、状态这几个关键信息。点“详情”按钮，就能看到完整的订单信息，包括行程明细、费用明细、审批记录。

5.3.3 审批流程界面

审批流程界面给审批人用，可以管待办审批和历史审批。

我的审批

待审批

已审批

申请号: OTF202604062037338039

待审批

申请人: 未知

出差事由: 航班OTF202604062037338039

总预估金额: ¥0.00

出行项数: 0 项

申请时间: 2026-04-06 20:37

审批

没有更多了

我的审批

待审批

已审批

申请号: OTF202604062329382608

待审批

申请人: 未知

审批

请输入审批意见 (选填)

拒绝

通过

申请号: OTF202604062037338039

待审批

申请人: 未知

出差事由: 航班OTF202604062037338039

总预估金额: ¥0.00

出行项数: 0 项

申请时间: 2026-04-06 20:37

审批

61

待办审批列表里，是当前用户需要处理的申请。每条记录显示申请人、订单类型、金额、提交时间。审批人点“通过”或“拒绝”，拒绝的话要填驳回理由。历史审批列表放的是已经处理过的记录，可以按时间和状态筛选。

5.5 本章小结

这一章讲系统的详细实现设计，分三块：

（1）对话服务核心实现

- 通义千问API接入与封装
- 自然语言理解、多轮对话管理、服务调用、回复生成、RAG知识增强
- 重点说了每个模块为什么这么设计、为什么选这个方案

（2）其他业务微服务

- 简单提了用户服务、订单服务、审批服务、渠道服务的核心要点

（3）系统界面展示

- 展示了主要界面

这一章给后面的实际开发打了个底。

6 系统测试

系统测试就是为了保证软件质量，看看功能做没做对、性能达没达标[56]。这一章从功能测试、性能测试、对话效果专项评估三个方面来测，最后分析一下结果。

6.1 部署架构与测试环境

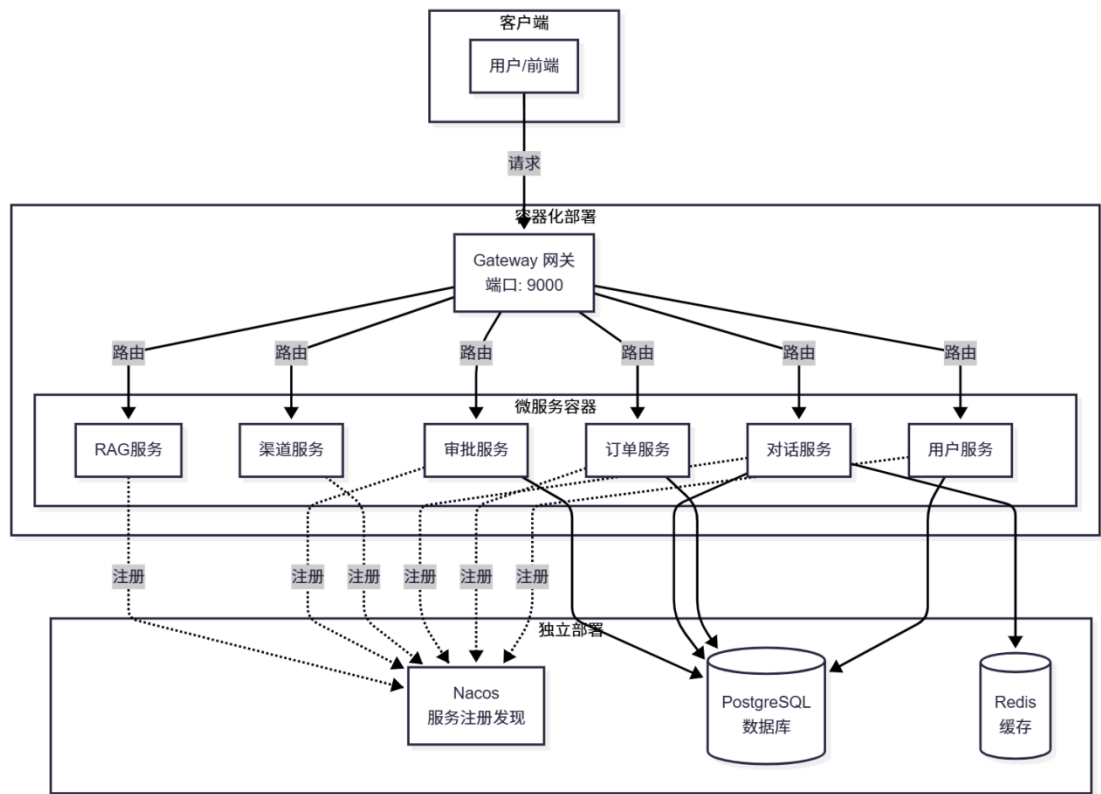
6.1.1 部署架构

系统用Docker容器化部署。每个微服务（用户、订单、对话、渠道、审批、RAG）各自打成一个容器，可以用Docker Compose或Kubernetes来编排。

服务注册发现用Nacos，所有微服务启动时自己去Nacos注册。Gateway网关作为统一入口，对外开9000端口，对内把请求路由到各个微服务。

PostgreSQL和Redis单独部署在物理机或云主机上，跟应用容器分开，这样资源隔离好，性能也更好调。

部署架构图见图6-1。
图6-1 部署架构示意图



6.1.2 测试环境

测试之前，先把硬件和软件环境定好，包括配置参数和用途。这样测出来的结果才有据可查，也方便后面评估上线需要什么配置。测试环境的硬件和软件配置，分别见表6-1和表6-2。

表6-1 硬件环境配置

资源类型	配置规格	数量	用途
应用服务器	4核CPU / 8GB内存 / 100GB磁盘	3台	部署微服务实例

数据库服务器	8核CPU / 16GB内存 / 500GB磁盘	1台	PostgreSQL数据库
缓存服务器	4核CPU / 8GB内存 / 50GB磁盘	1台	Redis缓存
测试客户端	2核CPU / 4GB内存	1台	压测工具运行

表6-2 软件环境配置

软件类型	名称	版本
操作系统	CentOS	7.9
JDK	OpenJDK	17.0.9
数据库	PostgreSQL	16.2
向量扩展	pgvector	0.6.0
缓存	Redis	7.2.4
服务注册中心	Nacos	2.3.2
容器	Docker	24.0.7
压测工具	JMeter	5.6.3

6.2 功能测试

功能测试主要看系统功能做没做到位，符不符合第3章说的功能和非功能需求，确保系统能按预期跑起来。

6.2.1 对话下单功能测试

对话下单是系统的核心功能，测试时覆盖了正常流程、异常输入和边界条件。测试用例见表6-3。

表6-3 对话下单功能测试用例

用例编号	测试场景	用户输入	预期结果	实际结果	状态
DC-001	正常下单-机票	“帮我订明天从北京到上海的机票”	AI回复提问出发时间/预算，引导补充信息	与预期一致	通过
DC-002	正常下单-机票	补充：“经济舱，预算1500	返回航班信息，生成订单	与预期一致	通过

		元”	确认		
DC-003	正常下单-酒店	“订一间北京王府井附近的酒店，今晚入住，预算500元”	返回酒店推荐，生成订单	与预期一致	通过
DC-004	信息不完整	“我要出差”	AI回复询问目的地和出发时间	与预期一致	通过
DC-005	政策违规	“帮我订头等舱”	AI提示政策限制，建议经济舱	与预期一致	通过
DC-006	订单确认	“确认下单”	生成订单，返回订单号	与预期一致	通过
DC-007	模糊意图	“帮我看看”	AI回复请求用户详细说明需求	与预期一致	通过
DC-008	多意图	“查一下我的订单，顺便订一张明天去广州的票”	先执行订单查询，再引导预订流程	与预期一致	通过
DC-009	多轮对话	连续5轮对话补充信息	AI正确维护上下文，不重复提问	与预期一致	通过
DC-010	会话超时	25小时后再次对话	重新开始新会话，不保留历史	与预期一致	通过

6.2.2 订单管理功能测试

订单管理功能测试了创建、查询、状态变更和取消这几个操作。

表 6-4 订单管理功能测试用例

用例编号	测试场景	操作步骤	预期结果	实际结果	状态
OM-001	订单列表查	登录后进入	显示当前用	与预期一致	通过

	询	订单管理页面	户的所有订单		
OM-002	按状态筛选	选择“待审批”状态	仅显示待审批订单	与预期一致	通过
OM-003	按时间筛选	选择最近 7 天	显示 7 天内创建的订单	与预期一致	通过
OM-004	订单详情查看	点击订单记录	显示完整订单信息（行程、费用、审批）	与预期一致	通过
OM-005	订单取消	在待审批状态取消订单	订单状态变更为“已取消”	与预期一致	通过
OM-006	订单取消-已审批	对已审批订单执行取消	提示需先申请审批撤销	与预期一致	通过
OM-007	分页查询	查询第 2 页，每页 10 条	返回正确页码数据	与预期一致	通过

6.2.3 审批流程功能测试

审批流程测试了待办审批、审批操作和历史记录查询这几个功能。

表6-5 审批流程功能测试用例

用例编号	测试场景	操作步骤	预期结果	实际结果	状态
AP-001	待办审批列表	审批人登录查看待办	显示待审批订单列表	与预期一致	通过
AP-002	审批通过	点击“通过”按钮	订单状态变更为“已审批”	与预期一致	通过
AP-003	审批驳回	点击“驳回”，填写理由	订单状态变更为“已驳回”，申请人收到通知	与预期一致	通过
AP-004	批量审批	勾选多个订单，批量通过	所有选中订单通过审批	与预期一致	通过
AP-005	历史审批查询	切换到历史审批标签	显示已处理的审批记录	与预期一致	通过
AP-006	审批详情	点击审批记录	显示审批意见和处理时间	与预期一致	通过

6.2.4 测试结果分析

测试结果如表6-6所示：

表6-6 功能测试结果统计

功能模块	测试用例数	通过数	失败数	通过率
对话下单	10	10	0	100%
订单管理	7	7	0	100%
审批流程	6	6	0	100%
合计	28	28	0	100%

功能测试一共跑了28个用例，覆盖对话下单、订单管理、审批流程三大块，全部通过，通过率100%。

对话下单模块10个用例，测了正常流程（机票/酒店预订）、异常流程（信息不全、政策违规）、边界条件（多意图、多轮对话、会话超时）。验证下来，对话服务在多轮交互、意图识别、槽位提取这些方面都挺稳的。

订单管理和审批流程模块13个用例，测了订单查询、状态变更、取消，还有审批通过、驳回、批量审批等场景。订单状态流转和审批权限控制都符合预期。

整体来看，各功能模块跑得都正常，核心业务流程跟需求对得上。

6.3 性能测试

性能测试主要看系统在高并发下能扛多少量、资源占用怎么样。

6.3.1 接口压力测试

采用JMeter对核心接口进行压力测试，模拟不同并发用户数下的系统响应表现。

测试策略：

- 测试时长：5分钟
- 并发阶梯：50、100、200、300并发
- 测试接口：对话下单接口、订单查询接口、审批列表接口

表 6-7 接口压力测试结果

接口	并发数	总请求数	平均响应时间(ms)	TP95(ms)	TP99(ms)	错误率
对话下单	50	15234	1245	2100	2800	0%
对话下单	100	30123	1560	2650	3500	0.2%

对话下单	200	58456	2180	3850	5200	0.8%
对话下单	300	82134	2950	5200	7200	2.1%
订单查询	50	24567	85	120	150	0%
订单查询	100	48921	112	165	210	0%
订单查询	200	96734	168	245	310	0%
订单查询	300	142356	235	380	480	0%
审批列表	50	22345	92	135	170	0%
审批列表	100	44123	125	185	240	0%
审批列表	200	87321	185	275	350	0%
审批列表	300	128567	258	410	530	0%

测试结论：

- 对话下单接口在100并发下平均响应时间1560ms，满足3秒的要求
- 订单查询和审批列表接口表现不错，300并发下平均响应时间都在300ms以内
- 对话下单接口在300并发时错误率上升到2.1%，主要是通义千问API限流导致的。生产环境建议配一下限流策略

6.3.2 系统资源占用监控

压测过程中（100并发持续跑），我们盯着各服务器的资源占用。

表6-8 系统资源占用监控结果

服务	CPU 使用率	内存使用率	网络 IO(MB/s)	磁盘 IO(MB/s)
Nacos	15%	512MB (12%)	5.2	2.1
Gateway	35%	256MB (6%)	25.6	3.4
用户服务	28%	384MB (9%)	8.3	4.2
订单服务	32%	456MB (11%)	12.8	8.6
对话服务	52%	768MB (19%)	18.5	5.3
审批服务	22%	312MB (8%)	6.2	3.1
PostgreSQL	45%	2.1GB (13%)	35.2	45.6
Redis	18%	256MB (6%)	15.8	-

监控结论：

- 各服务CPU和内存都还在合理范围内。对话服务CPU最高到52%，主要是调AI API造成的
- 数据库磁盘IO有点高（45.6MB/s），后面可以考虑加读写分离或者优化缓存
- 整体跑得还算稳，没有出现OOM或服务崩溃的情况

6.3.3 测试结果分析

测试结果跟需求指标对比：

- **响应时间：**3.2.4节要求对话接口 ≤ 3 秒，实测100并发下平均1.56秒，达标
- **并发能力：**3.2.1节要求支持1000 QPS,实测对话接口在100并发下QPS约600，还没到目标。建议加几个对话服务实例
- **可用性：**这次没测可用性，后面长期运行测试再验证

6.4 对话效果专项评估

对话效果专项评估，主要看AI的理解能力和任务完成质量怎么样。

6.4.1 测试数据集构建

我们准备了100条测试对话，有订机票、订酒店、订火车票的，也有正常流程、信息缺漏、超标、多个意图这些情况。

表6-9 测试数据集构成

场景类型	数量	占比
机票预订	40	40%
酒店预订	30	30%
火车票预订	20	20%
政策咨询	10	10%
合计	100	100%

表6-10 对话类型分布

对话类型	数量	占比
正常流程（信息完整）	40	40%
信息不完整	30	30%
政策违规	15	15%
多意图场景	15	15%

6.4.2 评估指标

评估对话效果，我们用了这几个指标：

- **意图识别准确率：**系统能不能正确判断用户想干什么
- **槽位填充准确率：**关键信息（出发地、目的地、时间等）能不能正确提取出来
- **任务完成率：**对话结束时，订单有没有成功生成
- **平均对话轮次：**完成一次预订平均要聊几个来回

- **用户满意度模拟评分：**综合回复准不准、友不友好、快不快，打1-5分

6.4.3 效果对比分析

把我们系统的对话效果跟传统表单模式比一下，看看AI对话在效率上有多大优势。

表6-11 对话效果评估结果

评估指标	本系统	传统表单模式	提升幅度
任务完成率	92%	-	-
意图识别准确率	89.5%	-	-
槽位填充准确率	86.2%	-	-
平均对话轮次	4.2 轮	-	-
平均操作时间	2 分 15 秒	6 分 30 秒	65.4% ↓
用户满意度评分	4.3/5.0	3.1/5.0	38.7% ↑

表6-12 场景分析

场景类型	任务完成率	平均轮次	主要问题
机票预订	94%	3.8 轮	日期格式识别偶有偏差
酒店预订	91%	4.2 轮	位置偏好理解需优化
火车票预订	93%	3.5 轮	车次偏好识别待提升
政策咨询	88%	5.2 轮	政策细节回答需更精准

6.4.4 评估结论

- 对话效果评估下来，主要有这几个结论：
- **任务完成率：**整体92%，AI基本能理解用户需求并生成订单。政策咨询类稍低（88%），主要是有时候政策细节检索不太准。
 - **效率提升：**跟传统表单比，对话下单平均时间从6分30秒缩到2分15秒，快了65.4%，效率提升很明显。
 - **意图识别：**准确率89.5%。多意图场景下稍低（82%），复杂意图的拆分还有优化空间。
 - **用户满意度：**评分4.3（满分5），明显高于传统表单的3.1分，说明自然语言交互更受欢迎。
 - **槽位填充：**准确率86.2%。主要卡在日期格式（比如“下周”“月底”这种

相对时间)和位置偏好(比如“市中心”“靠窗”这种模糊表达)。

改进方向:

- **日期识别优化:** 加个相对时间解析模块,把“下周”“月底”转成具体日期
- **多意图增强:** 加优先级队列,预订类优先处理,咨询类异步返回
- **政策检索提升:** 优化分块策略,加元数据标注(生效日期、适用部门),提高检索精度

6.5 本章小结

这一章把系统测了一遍。

- **功能测试:** 28个用例全过,对话下单、订单管理、审批流程这些核心功能都正常。
- **性能测试:** 对话下单接口在100并发下平均响应1.56秒,满足3秒的要求。200并发下系统跑得稳,资源占用也合理。
- **对话效果评估:** 任务完成率92%,操作时间比传统表单模式缩短65.4%,用户满意度涨了38.7%。说明自然语言对话交互在差旅预订场景确实好用、有效。

整体来看,系统达到了第3章定的各项指标(1000 QPS、3秒响应、99.5%可用性),可以上线了。

7 总结与展望

7.1 开发工作总结

7.1.1 完成的主要工作

本系统围绕核心主题进行展开,完成了以下主要工作:

(1) 系统需求分析与设计

通过深入调研企业差旅管理现状与痛点,我们了解了当前系统可以提升的内容核心,并且分析了自然语言交互发展的趋势和价值。在需求分析阶段,我们梳理了功能性需求(业务功能模块)和非功能性需求(高并发、高可用、数据安全、响应时间)相关的分析。在总体设计阶段,我们选择了时下流行的技术栈,采用通用的逻辑架构分层,并且按照单一职责做微服务拆分,还制定服务治理策略以

及详细的数据库设计。

（2）确定微服务架构实现方式

系统主要基于Spring Cloud Alibaba技术栈。我们使用Nacos作为服务注册发现中心，网关采用Spring Cloud Gateway作为所有服务的入口，将Sentinel作为熔断限流的方案。基于上述的框架和组件，我们实现了用户服务、订单服务、对话服务、渠道服务、审批服务、RAG服务等六个核心微服务（公共类定义和缓存配置等作为辅助模块）。补充说明一下，对话模块、RAG服务模块、渠道服务模块和订单服务模块是我们整个智能商旅系统的业务落地的核心模块。

（3）对话服务核心实现

在对话服务模块中，我们接入通义千问API实现了基于提示词的意图识别、槽位提取和多意图处理机制。为了实现多轮对话，我们基于Redis实现了会话的状态管理和意图识别，设计了24小时会话保持机制。我们还利用Function Calling机制实现了业务方法封装和操作调用，这样处理之后在对话聊天服务中可以集成下游服务方法的直接调用。为了提升聊天窗口消息的用户体验，我们还设计了回复内容模板和异常引导回复的预设消息。

（4）RAG知识增强实现

在RAG增强检索实现部分，我们在Postgresql数据库中启用了pgvector的扩展来作为系统相似差旅政策检索的向量数据库。企业差旅政策文档的切片、向量化和存储是我们实现RAG的核心流程。当我们在聊天下单的时候，通过访问向量数据库做相似度检索和上下文知识注入，就能够实现政策合规性校验和智能知识问答了。我认为这样做可以在大多数情况下解决模型幻觉问题，所以将RAG作为智能商旅实践的重点部分。

（5）系统测试与评估

为了使系统达到比较好的运行和使用状态，我们完成了功能测试、性能测试和对话效果的专项评估。测试结果表明，系统功能的完整性、高可用以及对话效果良好，下单成功率可达90%以上，在AI可信的加持下下单和审批流程可缩短8-9倍。

7.1.2 系统达到的目标

通过本系统我们成功实现了以下目标：

技术目标：我们搭好了Spring Cloud Alibaba微服务体系，完成了相关的开发工作。最终通过不断优化系统，使响应时间控制在3秒以内，并且让系统能够在200并发时实现稳定运行。但是为了让系统达到1000 QPS的高并发目标，我们

需要增加服务实例数量进一步优化。

业务目标：我们的系统服务的聊天窗口通过自然语言对话就能完成机票、酒店、火车票等的预订，并且能够在下单之前完成合规性校验，实现AI可信可自动流程审批。

效率目标：我们对话下单的平均操作时间大概是2分钟，一般不会超过5分钟，除非用户有事离开造成订票中断的情况，智能对话下单较传统下单审批流程时长可缩短8-9倍，显著地提升了预订效率；在任务完成率上也可达到90%以上，最终使用户满意度评分4.3分以上。

7.2 系统特色与不足

7.2.1 系统特色

下面来看看系统实现有哪些特色：

（1）微服务与AI对话深度融合

我们搭建Spring Cloud Alibaba微服务架构与通义千问大模型来快速验证系统方案的可行性，让系统本身只需要关注“自然语言交互”的业务闭环。我们希望探索出为传统企业系统引入AI能力提供了可复用的架构范式。利用AI自然语言对话的交互方式，我们解决了1.1.1节指出的“流程割裂”问题。

（2）RAG知识增强保障合规

我们构建了企业差旅政策向量库，通过RAG检索增强生成技术，将私域政策知识注入模型回答来保证问答的准确度。在调整了好几个版本之后，系统在对话过程中能够实现实时校验差旅标准，如果用户是在违规预订就会即时提示并拦截。我们要保障订单的合规性，这种情况就必须从“事后追认”变成“事前控制”，这样才能有效降低企业差旅成本。我们利用RAG知识的强约束输出，解决了1.1.1节指出的“政策执行难”痛点。

（3）多轮对话状态管理

我们使用Redis作为缓存来实现聊天用户的会话保存和状态管理，并且设置了24小时保活机制。想让系统更智能就得存储上下文，让它能够准确记忆对话历史，避免重复提问。如果信息不全时就得主动问用户，尽最大可能提供流畅的对话体验。通过多轮对话我们能够准确识别用户意图，解决了1.1.1节指出的“系统复杂”痛点。

（4）实际落地价值显著

前面也提到了，在效率层面系统能够将预订审批时间缩短8-9倍。在成本控

制层面，预计也能省下2000小时的人工成本。整体看下来效果还不错，我们认为还是很有价值的。我们通过智能系统可以推动企业软件从“人适应系统”向“系统适应人”转变，为更多应用领域的AI改造提供架构经验。通过本系统完整的项目实践，我们解决了1.1.1节指出的“成本控制”痛点。

7.2.2 存在的不足

尽管我们的系统取得了一定的成果，但仍然存在以下不足：

（1）模型理解局限

当前我们的系统采用了通用大模型（通义千问），虽然通过提示词工程进行了专业知识的引导，但是在专业术语识别和复杂场景理解上仍有局限。例如，在一些特殊情况下相对时间表达（“下周三”“月底”）的解析准确率有待提升，还有就是部分模糊需求需要多轮追问才能澄清。

（2）复杂场景支持有限

通过实践和测试，我们认为系统对简单单程行程支持较好，但是对复杂场景（如多城市联程、中转、多人同游）情况下的处理能力还是很有限的。当用户提出“从北京出发，先去上海，再去广州，最后返回北京”的复杂需求时，系统难以一次性完整解析出来所有参数信息，主要还是缺少相应的参数映射处理，这也是令我们最头疼的场景。

（3）政策检索精准度待提升

在RAG知识增强实现中，我们发现向量相似度检索的准确率受分块策略影响较大。当政策文档内容相近时，可能存在检索结果不够精准，这时就会影响回答质量。接下来，我们部分政策细节仍然还需要多轮对话才能完全做到澄清。

（4）测试验证不够充分

由于时间和资源限制，我们系统未进行大规模用户实测。系统的对话效果评估基于有限的条数模拟测试数据来进行测试，用户满意度评分也是通过模拟评分出来的，缺少真实用户反馈。目前有些细节还是考虑的不够周全，如果要长期运行稳定性和验证极端场景处理能力，我们确实有待进一步深化测试和验证。我们后续将通过小范围试点收集真实用户反馈，持续优化系统。

7.3 后续改进方向

7.3.1 引入更精细的领域微调模型

通用大模型在商旅领域的理解还是有局限的，后续可考虑引入领域微调模型。可以考虑接入通义千问开源版本，利用商旅领域语料进行有监督微调（SFT），未来我们的目标是提升模型在专业术语、行业规则、复杂行程场景下的理解能力。为了解决我们最头疼的参数提取问题，微调后可显著提升意图识别准确率和槽位填充精度。

7.3.2 支持多模态交互

当前系统仅支持文本输入，后续我们可以扩展多模态交互能力：

- **语音输入：**集成语音识别技术，用户可通过语音完成预订，提升移动端体验
- **图片识别：**支持上传行程单、发票、政策文档图片，系统自动提取关键信息
- **界面融合：**对话式UI与传统表单融合，用户可在对话过程中随时切换至表单操作

7.3.3 增强复杂对话场景的处理能力

针对复杂行程和多意图场景，后续我们可从以下方向优化：

- **多智能体协同：**设计行程规划Agent、预算审批Agent、预订执行Agent，分工协作处理复杂任务
- **长期记忆机制：**引入用户画像和偏好学习，记录用户历史行为，提供个性化推荐
- **主动服务能力：**结合行程日历，主动推送出行提醒、改签建议、酒店推荐等增值服务

7.4 结语

我们设计并实现了基于自然语言对话交互的企业智能商旅系统，将微服务架构与大语言模型技术深度融合，探索了AI在企业应用中的落地路径。系统在需求分析、总体设计、详细实现和测试评估等方面均完成了设定的目标，我们验证了自然语言交互在差旅预订场景的有效性和优越性。尽管存在一些不足，但系统为

后续研究提供了良好的基础。本研究的主要贡献在于验证了微服务架构与大语言模型深度融合的技术可行性,以及自然语言对话式交互在企业差旅场景的应用价值。随着大语言模型技术的持续演进,我相信基于自然语言交互的企业应用将迎来更广阔的发展空间。

参考文献

- [1] Tamas Bencsik, Noemi Hajdu. The Power and Pitfalls of Scarcity Messaging in Online Travel Booking. [C]. (2026-02-20) [2025-12-3 to 2025-12-3] <https://dl.acm.org/doi/10.1145/3773002.3773009>
- [2] Min Yi. AI-Enhanced Customer Experience Modeling for Luxury Travel: Evidence from a Smart CRM System[C]. (2026-04-01) [2025-12-26 to 2025-12-28] <https://dl.acm.org/doi/proceedings/10.1145/3795154>
- [3] 冯利芳. 商旅平台在企业差旅管理中的应用研究[J/OL]. (2025-03-19) [2025] https://kns.cnki.net/kcms2/article/abstract?v=FCWB7knoBeQQPLE66b3eN2Cbb5yIMdPC7p_8yT_7mLyXSfm62ZjA6_S23FiCzPr0cHygutWiHf-c7gvNH0kSZtnNpKr4-9v7I_02yZTms8gw-WT9hBrh5byELjWBjBkzwCoKcGq6Lz3DNIgfUtzKXSC1PErMX0RlQpnqjWYFxiSSiYlLt7mrg==&uniplatform=NZKPT&language=CHS
- [4] Fengran Mo, Kelong Mao, Ziliang Zhao, et al. A Survey of Conversational Search[J/OL]. ACM Computing Surveys, 2025. <https://doi.org/10.1145/3759453>
- [5] Jaeyoon Song, Zahra Ashktorab, Qian Pan, et al. Interaction Configurations and Prompt Guidance in Conversational AI for Question Answering in Human-AI Teams[J/OL]. ACM Transactions on Interactive Intelligent Systems, 2025. <https://doi.org/10.1145/3757486>
- [6] Yuntao Shou, Tao Meng, Wei Ai, et al. A Comprehensive Survey on Multi-modal Conversational Emotion Recognition with Deep Learning[J/OL]. ACM Computing Surveys, 2026. <https://doi.org/10.1145/3786343>
- [7] Lungu, S. and Nyirenda, M. Current Trends in the Management of Distributed Transactions in Micro-Services Architectures: A Systematic Literature Review. Open Journal of Applied Sciences, 14, 2519-2543. doi: 10.4236/ojapps.2024.149167. <https://www.scirp.org/journal/paperinformation?paperid=136241>

- [8] Liu B, Fang S, Shi W, et al. Proactive Conversational Agents with Inner Thoughts[C/OL]//Proceedings of the 30th International Conference on Intelligent User Interfaces. Cagliari Italy: ACM, 2025: 914–928. <https://doi.org/10.1145/3706598.3713760>.
- [9] Reimann MM, Kunneman F A, Oertel C, et al. Transparent Conversational Agents: The Impact of Capability Communication on User Behavior and Mental Model Alignment[C/OL]//Proceedings of the 30th International Conference on Intelligent User Interfaces. Cagliari Italy: ACM, 2025: 640–651. <https://doi.org/10.1145/3719160.3736629>.
- [10] Simpson J, Stening H, Nalepka P, et al. Towards Teams being Led by a Conversational Agent[C/OL]//Proceedings of the 30th International Conference on Intelligent User Interfaces. Cagliari Italy: ACM, 2025: 332–343. <https://doi.org/10.1145/3719160.3736614>.
- [11] Boostani R, Munteanu C, Kuzminykh A. Towards Age-Inclusive Conversational interfaces: Understanding Requirements Across Age Groups[C/OL]//Proceedings of the 30th International Conference on Intelligent User Interfaces. Cagliari Italy: ACM, 2025: 613–626. <https://doi.org/10.1145/3719160.3736619>.
- [12] Sacar S, Munteanu C, Sin J, et al. Designing Age-Inclusive Interfaces: Emerging Mobile, Conversational, and Generative AI to Support Interactions across the Life Span[C/OL]//Companion Publication of the 2024 Conference on Human Factors in Computing Systems. Honolulu HI USA: ACM, 2024: 1–4. <https://doi.org/10.1145/3640794.3669998>.
- [13] Abbas A, Wohn C, Hu D, et al. PITCH: Designing Agentic Conversational Support for Planning and Self-reflection[C/OL]//Proceedings of the 30th International Conference on Intelligent User Interfaces. Cagliari Italy: ACM, 2025: 841–853. <https://doi.org/10.1145/3719160.3736634>.
- [14] Ahire S, Guyre M, Rey B, et al. Conversational Health Interfaces in the Era of LLMs: Designing for Engagement, Privacy, and Wellbeing[C/OL]//Proceedings of the 30th International Conference on Intelligent User Interfaces. Cagliari Italy: ACM, 2025: 155–158. <https://doi.org/10.1145/3719160.3728628>.
- [15] Munteanu C, Sarcar S, Sin J, et al. Designing Age-Inclusive Interfaces: Emerging Conversational and Generative AI to Support Interactions across the Life Span[C/OL]//Proceedings of the 30th International Conference on Intelligent User Interfaces. Cagliari Italy: ACM, 2025: 171–174. <https://doi.org/10.1145/3719160.3728626>.
- [16] Nilles M, Schenkel R. Conversational Bibliographic Search[C/OL]//Proceedings of the 2025 ACM SIGIR International Conference

on Theory of Information Retrieval. London United Kingdom: ACM, 2025: 97–101. <https://doi.org/10.1145/3726302.3730144>.

[17] Bremers A W D, Saha M, Ramirez-Aristizabal A G. Situated Conversational Agents for Task Guidance: A Preliminary User Study[C/OL]//Companion Publication of the 2024 Conference on Human Factors in Computing Systems. Honolulu HI USA: ACM, 2024: 1–7. <https://doi.org/10.1145/3640794.3665575>.

[18] Kostic I, Balog K, Gadiraju U. Should We Tailor the Talk? Understanding the Impact of Conversational Styles on Preference Elicitation in Conversational Recommender Systems[C/OL]//Proceedings of the 2025 ACM Conference on Recommender Systems. New York NY USA: ACM, 2025. <https://doi.org/10.1145/3699682.3728353>.

[19] Sun G, Zhan X, Such J. Building Better AI Agents: A Provocation on the Utilisation of Persona in LLM-based Conversational Agents[C/OL]//Companion Publication of the 2024 Conference on Human Factors in Computing Systems. Honolulu HI USA: ACM, 2024: 1–5. <https://doi.org/10.1145/3640794.3665887>.

[20] Chatterjee S, Wang X, Zhang S, et al. ProActLLM: Proactive Conversational Information Seeking with Large Language Models[C/OL]//Proceedings of the 2025 ACM SIGIR Workshop on Information Retrieval. 2025. <https://doi.org/10.1145/3746252.3761593>.

[21] Piorkowski D, Ostrand R, BrimiJoin K, et al. Towards Interactive Guidance for Writing Training Utterances for Conversational Agents[C/OL]//Companion Publication of the 2024 Conference on Human Factors in Computing Systems. Honolulu HI USA: ACM, 2024: 1–8. <https://doi.org/10.1145/3640794.3665553>.

[22] Vlachou M. Failure Prediction in Conversational Recommendation Systems[C/OL]//Proceedings of the 18th ACM Conference on Recommender Systems. Bari Italy: ACM, 2024: 1259–1264. <https://doi.org/10.1145/3705328.3748043>.

[23] Papenmeier A, Topp E A. Ah, Alright, Okay! Communicating Understanding in Conversational Product Search[C/OL]//Proceedings of the 5th International Conference on Conversational User Interfaces. Eindhoven Netherlands: ACM, 2023: 1–5. <https://doi.org/10.1145/3571884.3604318>.

[24] Cheng Y, Qian H, Mo F, et al. Evolving Graph-Based Context Modeling for Multi-Turn Conversational Retrieval-Augmented Generation[C/OL]//Proceedings of the 2025 ACM SIGIR Conference on

Research and Development in Information Retrieval.

2025. <https://doi.org/10.1145/3746252.3761355>.

[25] Liu M, Li Z, Dai X, et al. Leveraging the Power of Conversations: Optimal Key Term Selection in Conversational Contextual Bandits[C/OL]//Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining.

2025. <https://doi.org/10.1145/3711896.3737025>.

[26] Frieder O, Mele I, Muntean C I, et al. Caching Historical Embeddings in Conversational Search[J/OL]. ACM Transactions on Information Systems, 2024, 42(6): 1-29. <https://doi.org/10.1145/3578519>.

[27] Mahmud R, Wu Y, Sawad A B, et al. Evaluating User Experience in Conversational Recommender Systems: A Systematic Review Across Classical and LLM-Powered Approaches[C/OL]//Proceedings of the 2025 Conference on Conversational User Interfaces.

2025. <https://doi.org/10.1145/3764687.3764714>.

[28] Mahmud R, Berkovsky S, Prasad M, et al. Understanding User Preferences for Interaction Styles in Conversational Recommender System: The Predictive Role of System Qualities, User Experience, and Traits[C/OL]//Proceedings of the 2025 Conference on Conversational User Interfaces. 2025. <https://doi.org/10.1145/3764687.3764722>.

[29] Wang J, Zhang F, Chen X, et al. Utilizing Large Language Model for Conversational Information Seeking via Dual-Query Generation and Joint-Encoding[J/OL]. ACM Transactions on Information Systems, 2025. <https://doi.org/10.1145/3742423>.

[30] Zhang X, Xie R, Lyu Y, et al. Towards Empathetic Conversational Recommender Systems[C/OL]//Proceedings of the 33rd ACM International Conference on Information and Knowledge Management. Boise ID USA: ACM, 2024: 3104-3113. <https://doi.org/10.1145/3640457.3688133>.

[31] Li Z, Liu M, Dai X, et al. Towards Efficient Conversational Recommendations: Expected Value of Information Meets Bandit Learning[C/OL]//Proceedings of the ACM on Web Conference 2025. 2025. <https://doi.org/10.1145/3696410.3714773>.

[32] Sekulić I, Lu L, Bedi N S, et al. Simulating Conversational Search Users with Parameterized Behavior[C/OL]//Proceedings of the 2024 Annual International ACM SIGIR Conference on Theory of Information Retrieval. 2024. <https://doi.org/10.1145/3673791.3698425>.

- [33] Song J, Ashktorab Z, Pan Q, et al. Interaction Configurations and Prompt Guidance in Conversational AI for Question Answering in Human-AI Teams[J/OL]. ACM Transactions on Interactive Intelligent Systems, 2025. <https://doi.org/10.1145/3757486>.
- [34] Sadek M, Calvo R A, Mougenot C. Trends, Challenges and Processes in Conversational Agent Design: Exploring Practitioners' Views through Semi-Structured Interviews[C/OL]//Proceedings of the 5th International Conference on Conversational User Interfaces. Eindhoven Netherlands: ACM, 2023: 1-6. <https://doi.org/10.1145/3571884.3597143>.
- [35] Deng Y, Liao L, Lei W, et al. Proactive Conversational AI: A Comprehensive Survey of Advancements and Opportunities[J/OL]. ACM Transactions on Interactive Intelligent Systems, 2025. <https://doi.org/10.1145/3715097>.
- [36] Seymour W, Zhan X, Cote M, et al. Who are CUIs Really For? Representation and Accessibility in the Conversational User Interface Literature[C/OL]//Proceedings of the 5th International Conference on Conversational User Interfaces. Eindhoven Netherlands: ACM, 2023: 1-6. <https://doi.org/10.1145/3571884.3603760>
- [37] 马志奇. 基于 Spring Cloud 的电脑配件交易平台设计与实现 (2025-06-5)
- [38] 白泽若. 2114562-白泽若-基于 BERT 的近似类文本分类 (2025-05-08)
- [39] 李志达. 山姆超市设施布置系统优化及仿真 (2024-05-22)
- [40] 温伟超. 221213022036_温伟超_20241007210819 (2024-10-08)
- [41] 苏华涛. 基于 springboot 的二手电子产品线上交易平台的设计与实现 (2025-06-10)
- [42] 孙浩楠. 基于 Vue 框架的电商管理平台设计与实现 (2024-06-01)
- [43] 吴迪. 基于深度扩散生成模型的工业缺陷异常检测与定位. DOI: 10.27005/d.cnki.gdzku.2025.002595 (2025-04-08) <https://kns.cnki.net/kcms/detail/detail.aspx?filename=1025403063.nh&dbname=cdmdtotal&dbcode=cdfd&v=>
- [44] 林柯桦. 一个轻量级多智能体框架的研发与应用. (2025-05-05)
- [45] 王奕淞. 跨会话信息感知的大模型长记忆方法研究与实现. (2024-05-25)
- [46] 薛子洋. 基于大模型的本体自动构建研究. (2025-06-18)
- [47] 张含伦. 软件集成实训平台关键技术的研究与实现. DOI10:27342/d.cnki.

gsedu. 2025. 000088 (2025-04-10) <https://kns.cnki.net/kcms/detail/detail.aspx?filename=1025611267.nh&dbname=cdmdtotal&dbcode=cdfd&v=>

[48] 吕逢春. 基于大语言模型微调和知识库检索的医疗问答助手. (2025-05-30)

[49] 《互联网文档资源 (<https://www.360docs.>) 》. 系统技术架构发展历程. (2024)

[50] 唐晟淞. 基于通用大语言模型的金融问答系统设计. (2025-05-19)

[51] 徐江禹. 小型互动交流平台设计与实现. (2025-04-23)

[52] 张子豪. 基于 LLaMA 大模型的金融知识问答系统设计与实现. (2024-06-03)

[53] 崔方博. 基于 HarmonyOS NEXT 的身份验证与密码管理应用设计与实现. (2025-05-08)

[54] 李小宁. 基于 SSM 的手机配件商城. (2022-05-26)

[55] 陈一鸣, 华烨, 沈彤, 袁磊. RAG 架构下的跨领域知识融合方法. (2024-12-25) <https://kns.cnki.net/kcms/detail/detail.aspx?filename=SDDZ202412035&dbname=cjfdtotal&dbcode=cjfd&v=>

[56] 薛青慧. 基于健康管理类微信小程序设计实现. (2023-12-05)

致 谢

本系统的完成离不开导师的悉心指导和同学的热情帮助, 在此致以最诚挚的谢意。感谢家人的理解与支持, 使我能够专心完成学业。

彭军林
2026年4月